

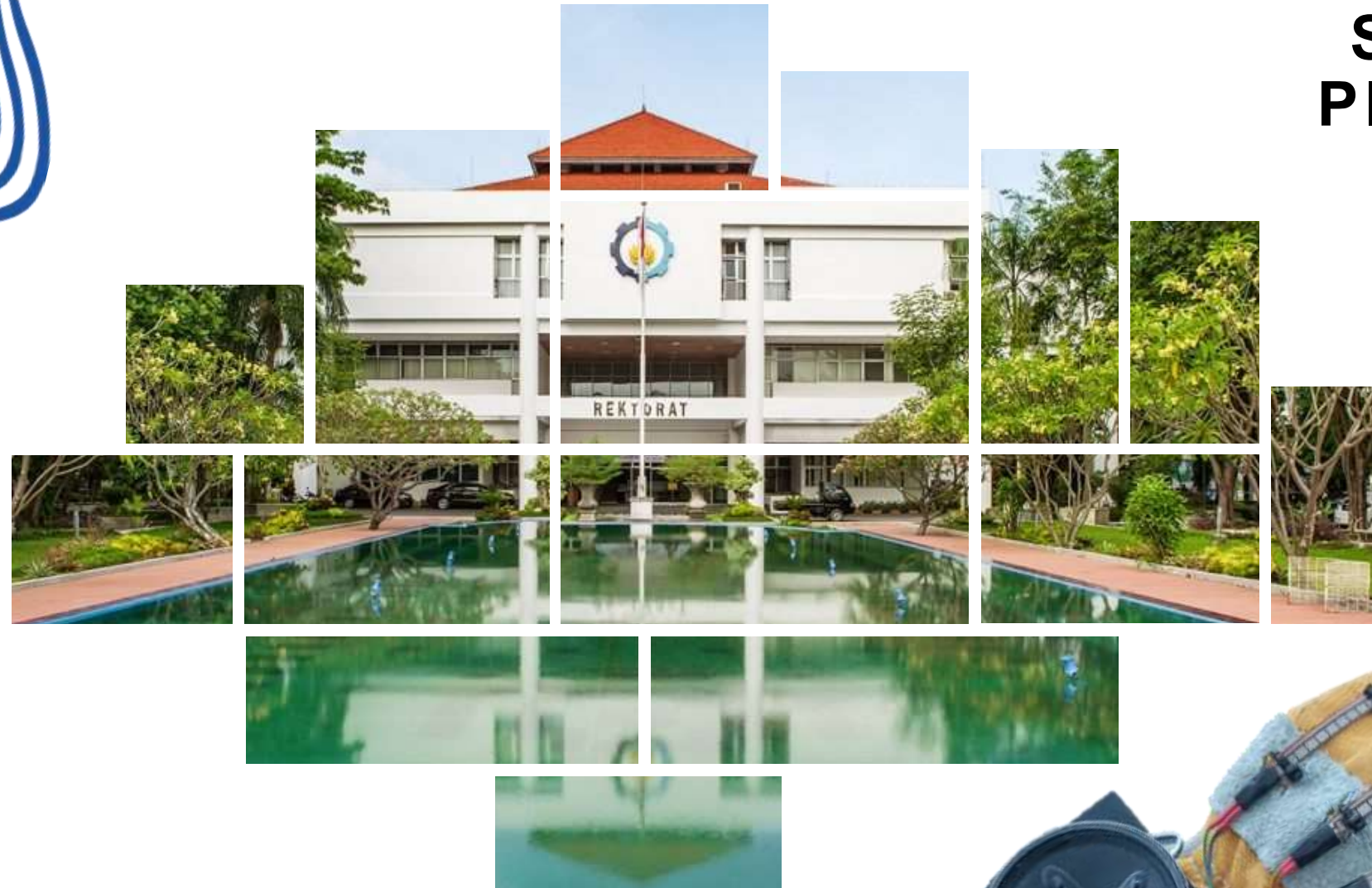
STUDI DAN PENGEMBANGAN SARUNG TANGAN PINTAR TUNAWICARA: MEMANFAATKAN SENSOR GYRO DAN *FLEX* DENGAN KLASIFIKASI DATA BERBASIS JARINGAN SARAF TIRUAN

Dibuat Oleh:
Muhammad Fauzan Ariyatmoko
NRP 5001211062

Dosen Pembimbing:
Diky Anggoro, S.Si., M.Si
NIP 198508092014041003

اللَّهُمَّ يَسِّرْ وَلَا تُعَسِّرْ.

“Allahumma yassir wala tu'assir”





LATAR BELAKANG



Menurut Badan Pusat Statistik populasi penyandang disabilitas tunawicara Jawa Timur pada tahun 2019 tembus 5.021 Jiwa.

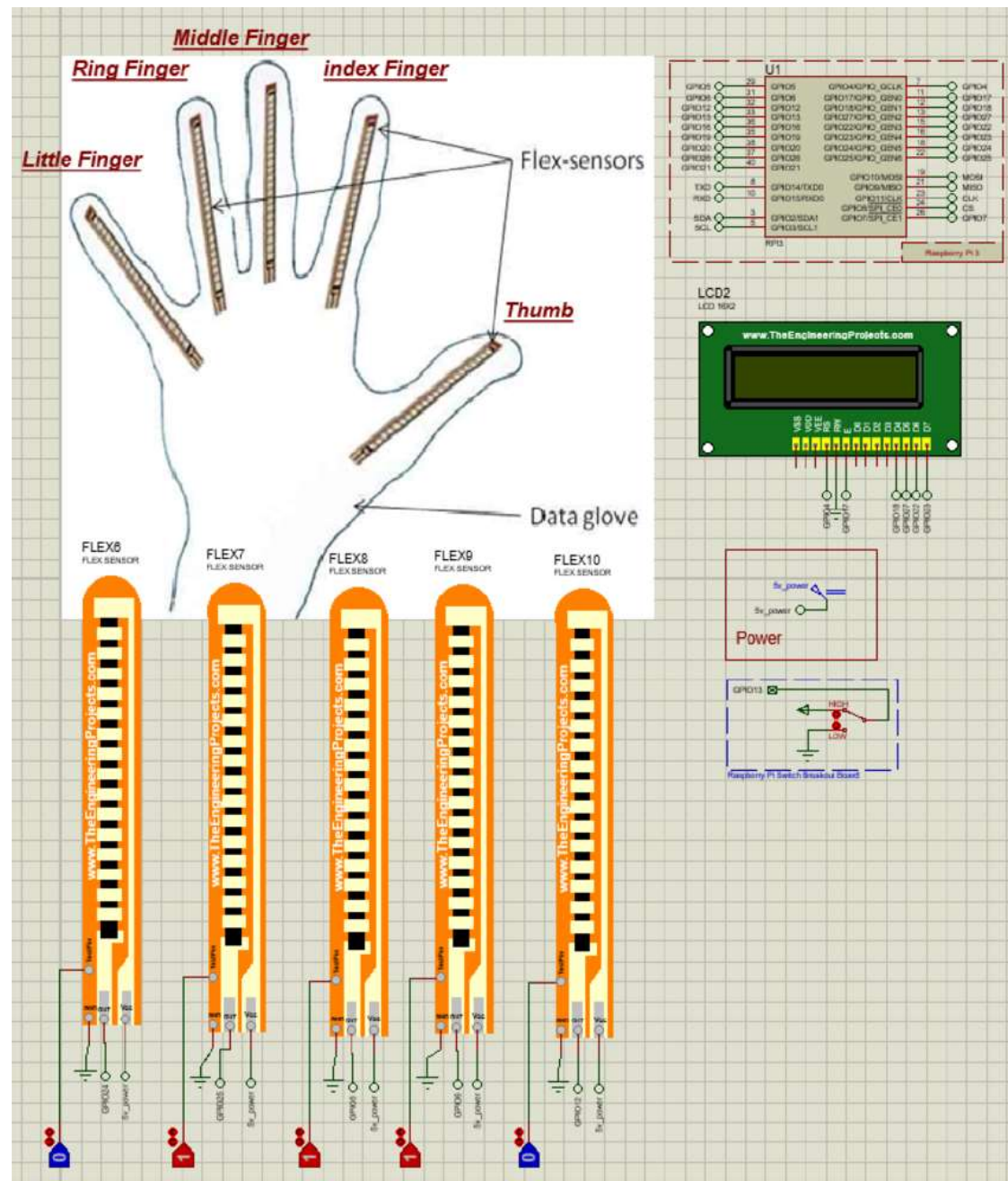
Penyandang disabilitas tunawicara seringkali sulit untuk berkomunikasi dan bersosialisasi kepada masyarakat.

Terbatasnya akses perangkat asistif (alat bantu) visual maupun non-visual bagi para penyandang disabilitas tunawicara di Indonesia.



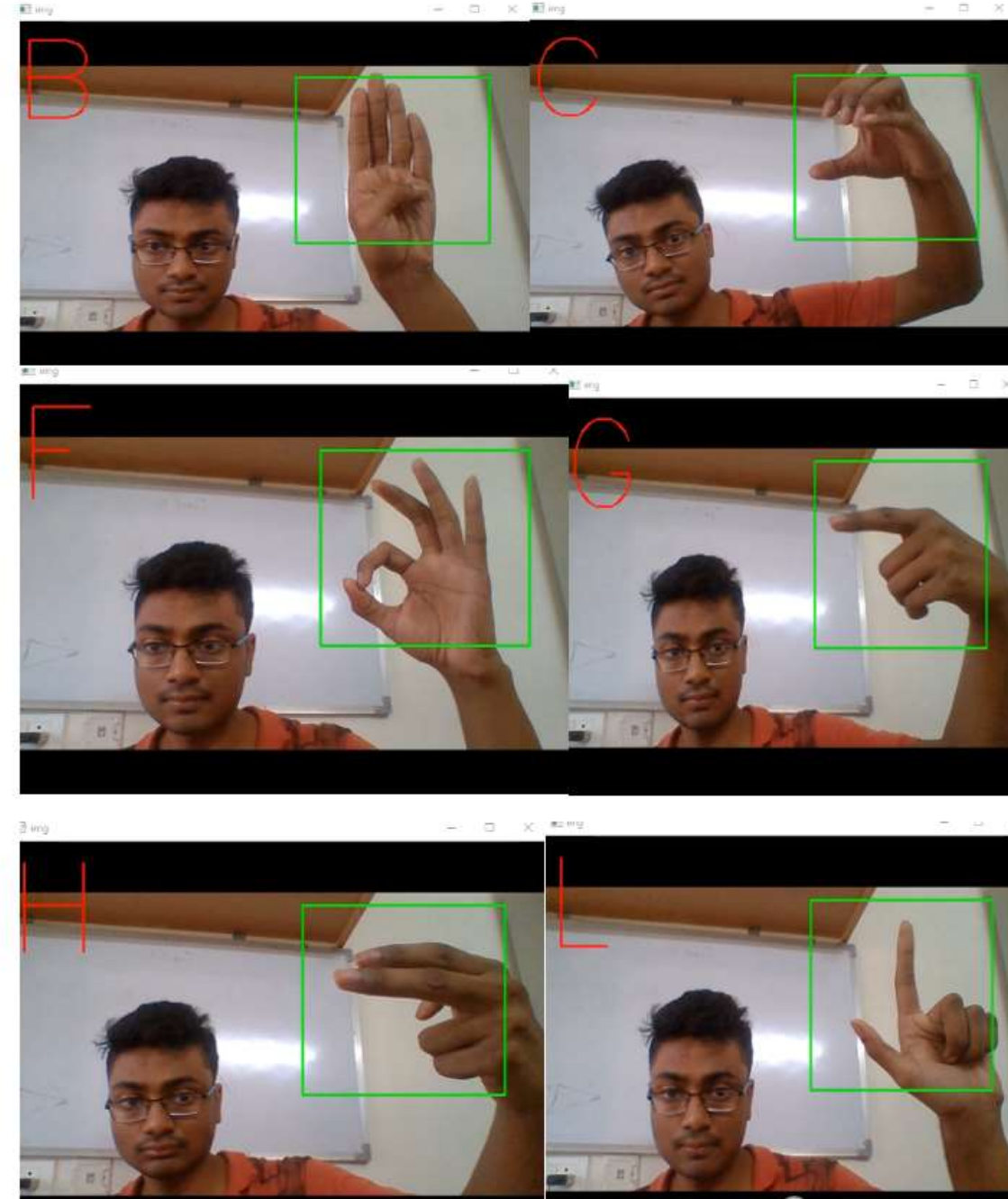
PENELITIAN TERDAHULU

Alat Bantu secara Non-Visual



Serhat Küçükdermenci, 2023.

Alat Bantu secara Visual



themechanicalcoder, 2021.

Percobaan *dummy* alat bantu Visual SIBI App





TUJUAN PENELITIAN

Mengembangkan sistem komunikasi penyandang disabilitas tunawicara terhadap masyarakat yang minim dalam ilmu bahasa isyarat.

Menganalisis prinsip kerja dan sifat fisis sensor flex dan Girooskop IMU MPU6050 pada sarung tangan pintar.

Membuat prototipe sarungan tangan pintar sebagai perangkat alat bantu penyandang disabilitas tunawicara.



BATASAN MASALAH

Sensor yang digunakan pada pembuatan alat bantu disabilitas sarung tangan pintar tunawicara adalah sensor giroskop IMU MPU6050 dan *Flex*.

Pada penelitian ini hanya difokuskan menerjemahkan Abjad SIBI dengan satu tangan, sehingga penerjemahan angka maupun sistem isyarat lain belum diterapkan.

Karakteristik material sensor tidak diteliti lebih lanjut sehingga tidak dibahas dalam penelitian ini.

TINJAUAN PUSTAKA

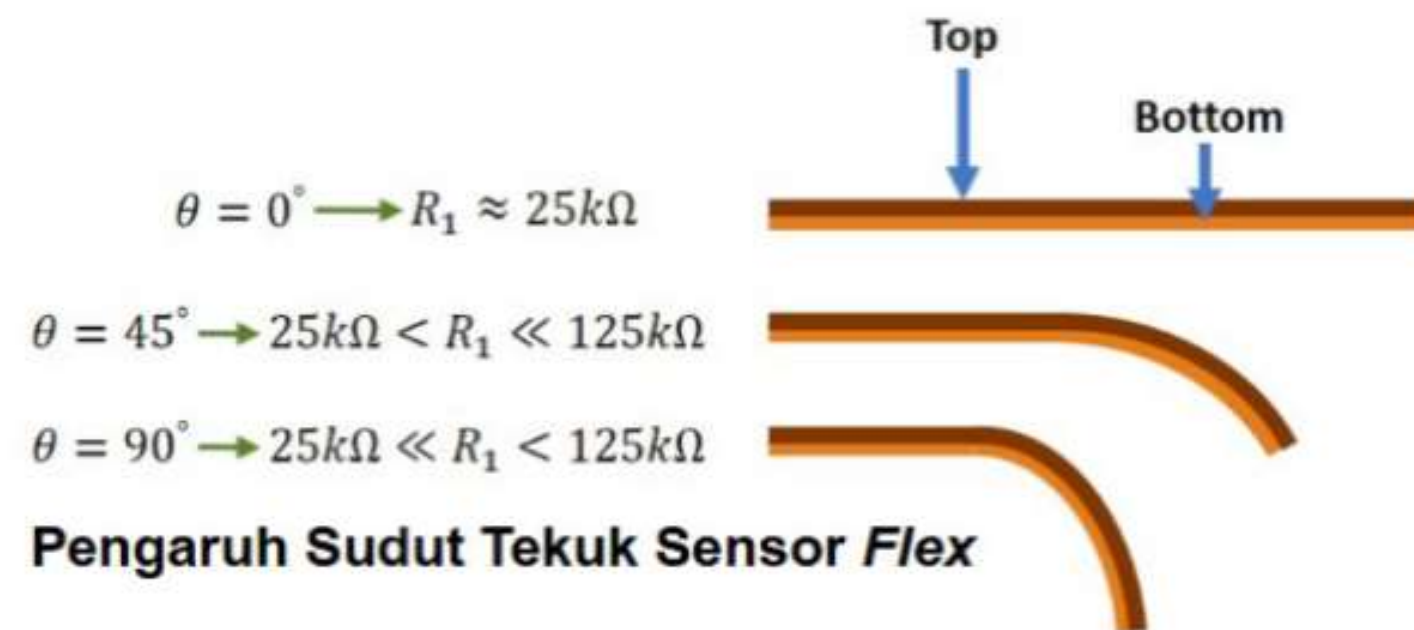
Sistem Isyarat Bahasa Indonesia (SIBI)



<https://pmpk.kemdikbud.go.id/sibi/kosakata>

TINJAUAN PUSTAKA

Sensor *Flex* (Resistor Fleksible)



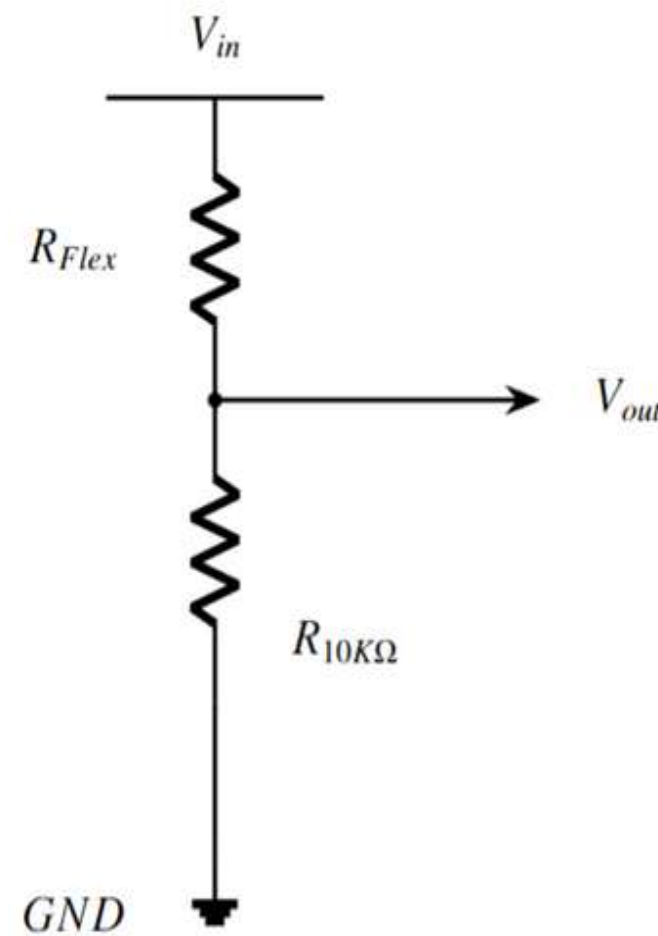
Syed Mubarak Ali et al., 2019



TINJAUAN PUSTAKA

Sensor *Flex* (Resistor Fleksible)

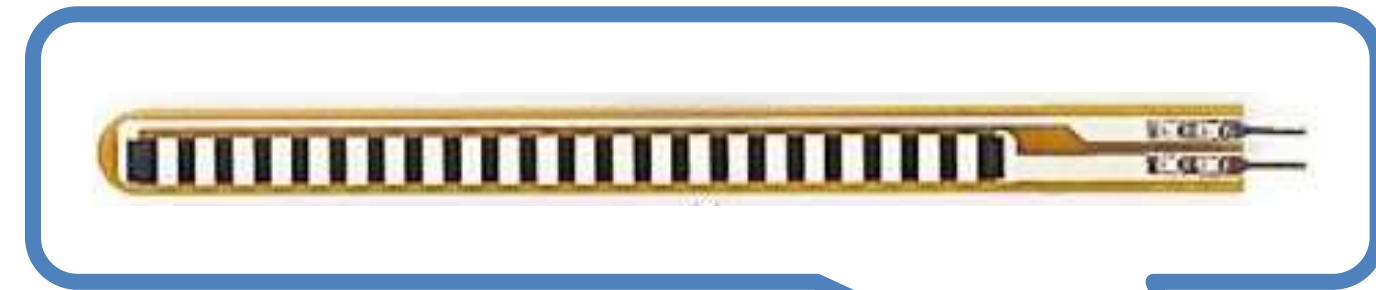
Rangkaian Pemasangan Sensor *Flex*



Pembagi Tegangan

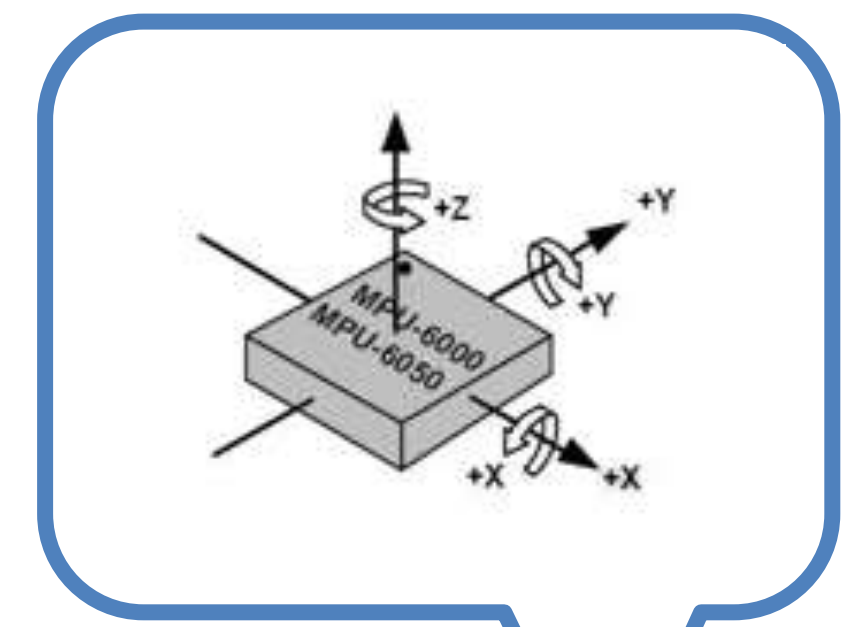
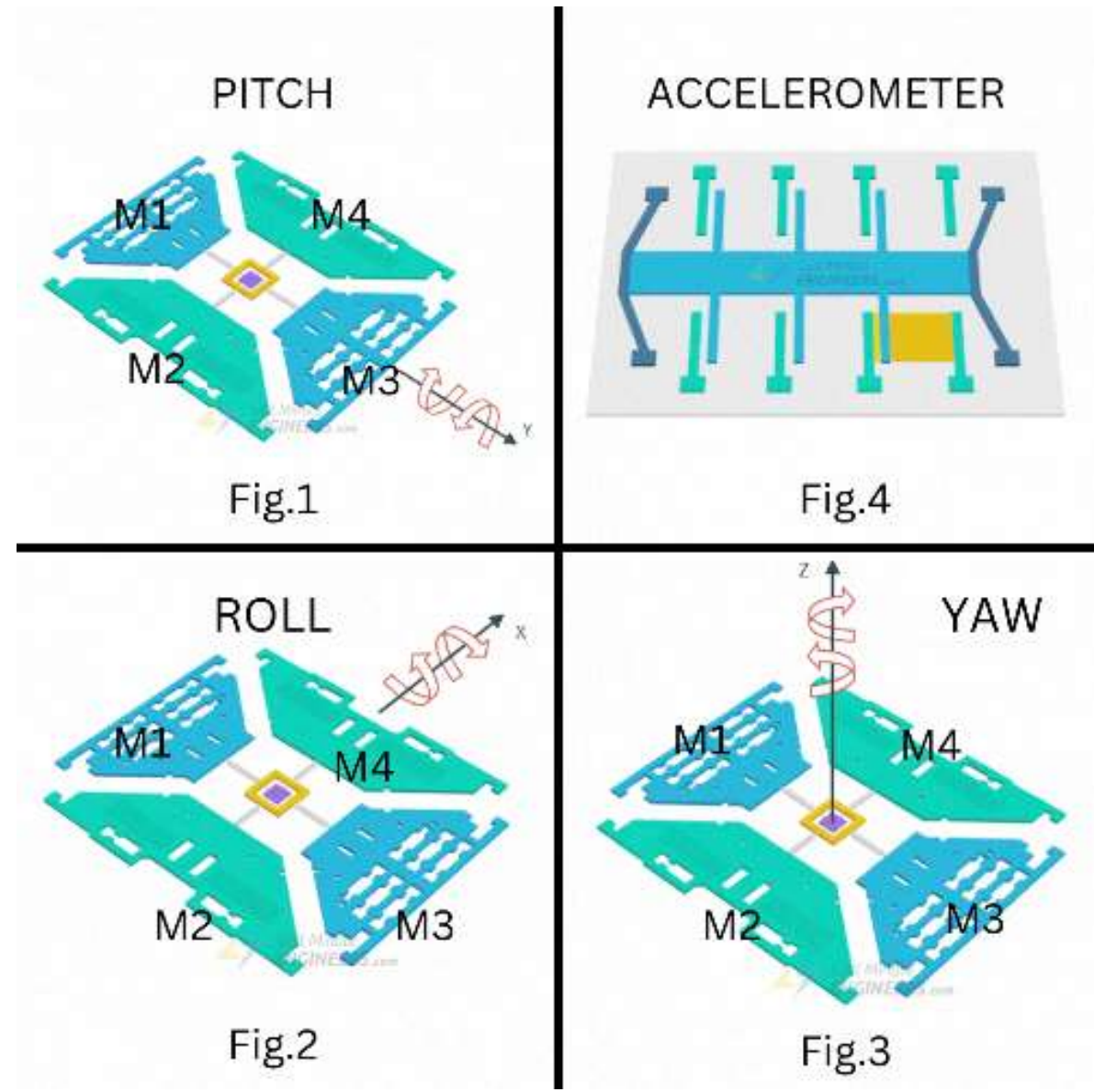
$$V_{out} = V_{in} \cdot \frac{R_{10K\Omega}}{R_{flex} + R_{10K\Omega}}$$

$$R_{flex} = R_{10K\Omega} \cdot \left(\frac{V_{in}}{V_{out}} - 1 \right)$$



TINJAUAN PUSTAKA

Sensor Giroskop (IMU MPU6050)

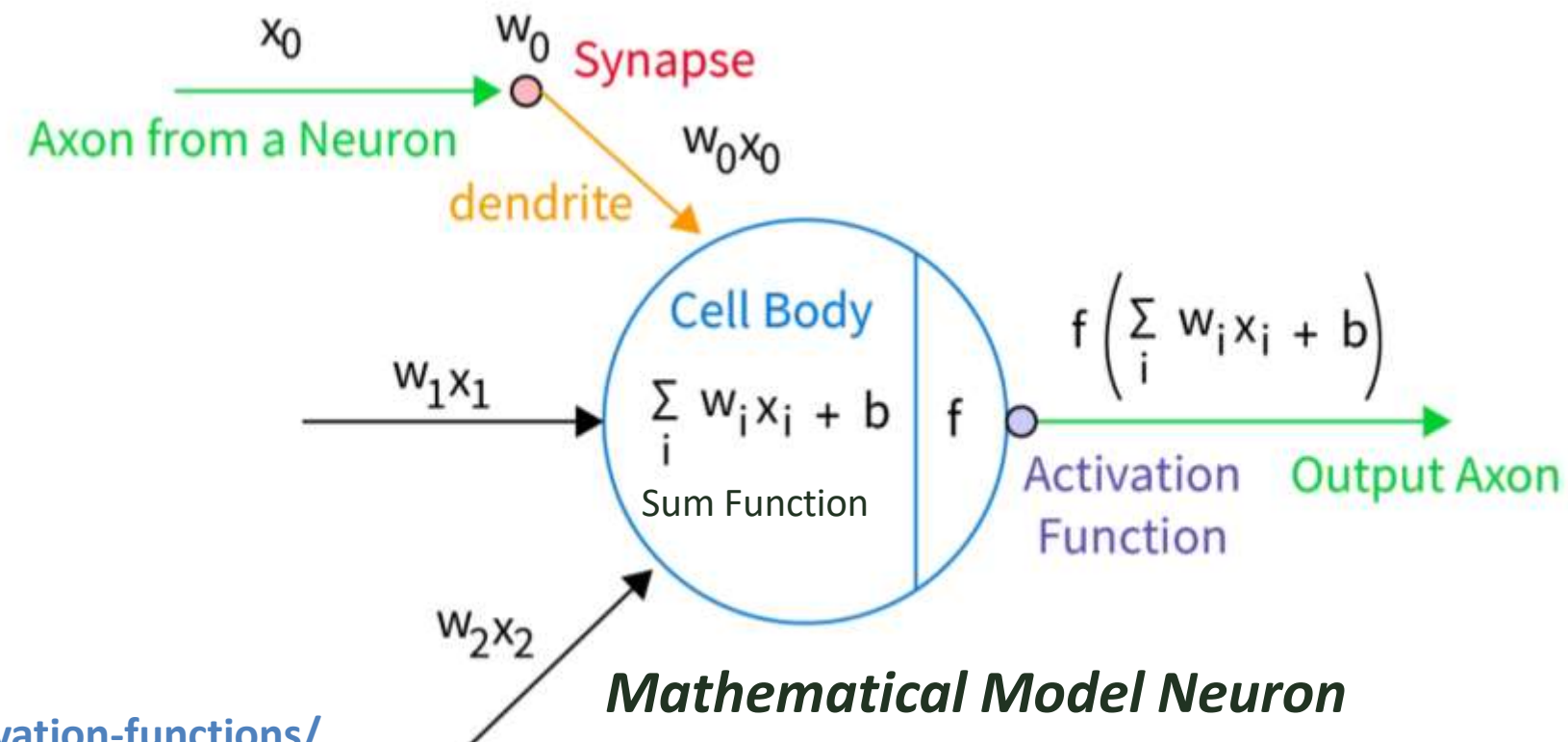
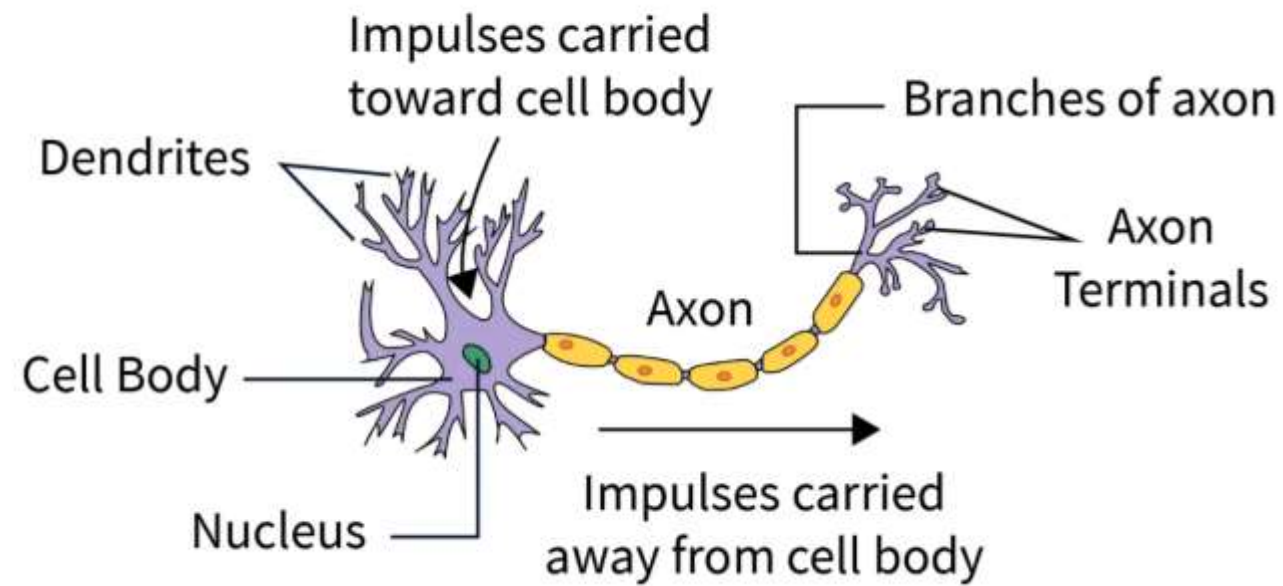


<https://forum.arduino.cc/t/getting-started-with-the-mpu6050-beginner-friendly-guide-with-visual-explanations>

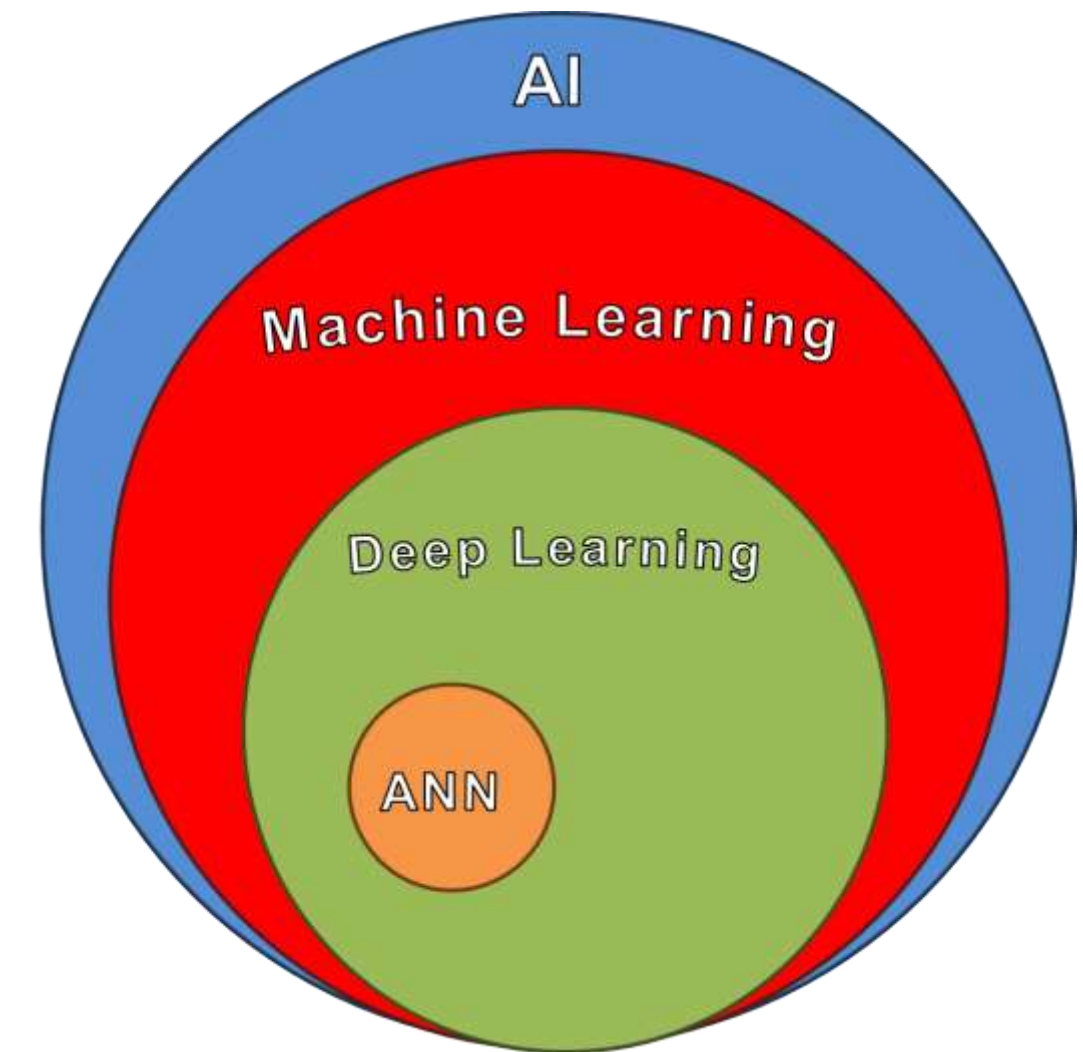
TINJAUAN PUSTAKA

Jaringan Saraf Tiruan (ANN)

Biological Neuron



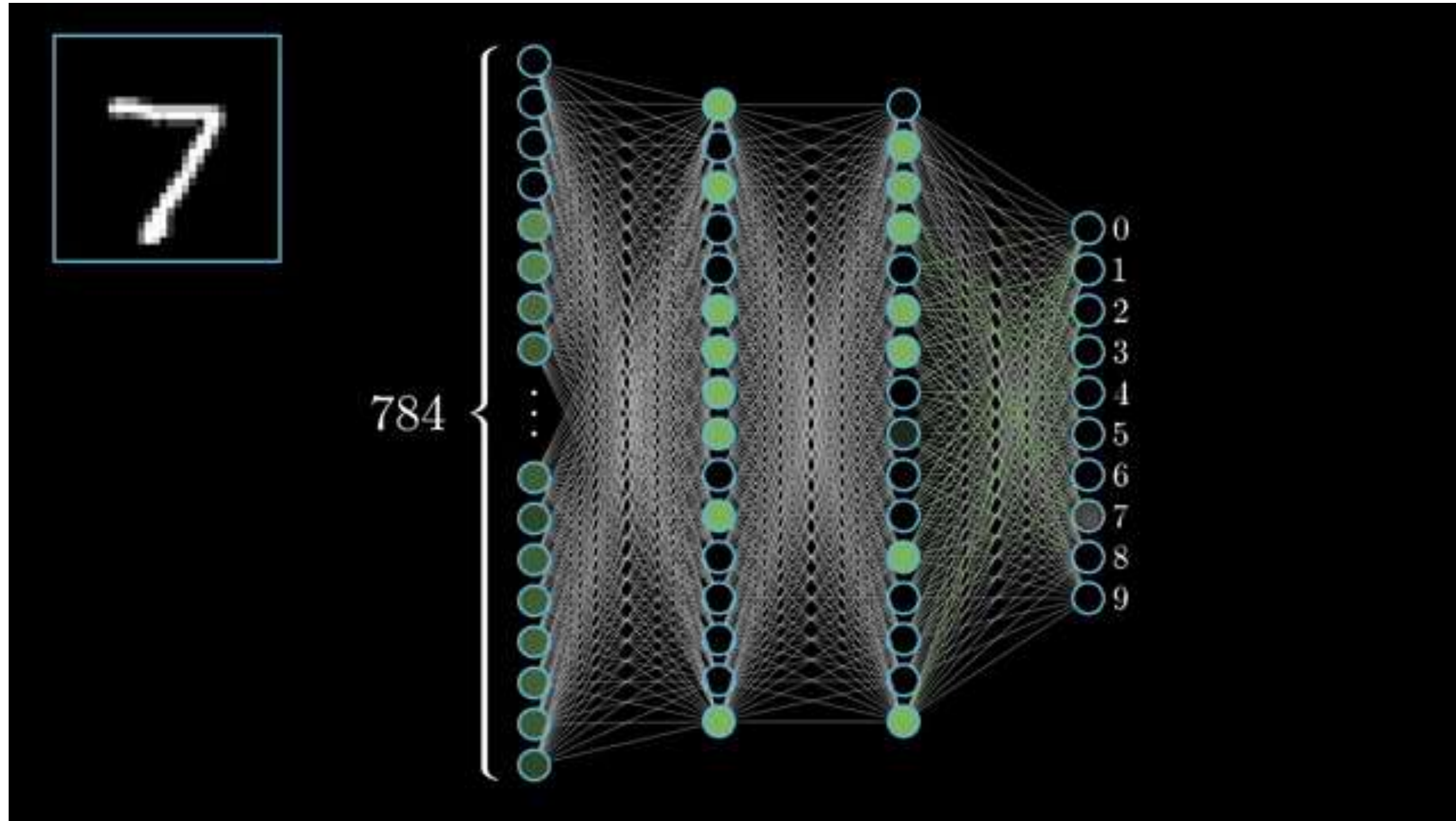
Mathematical Model Neuron



Lingkup Jaringan Saraf Tiruan (ANN)

TINJAUAN PUSTAKA

Contoh cara kerja Jaringan Saraf Tiruan (ANN)



TINJAUAN PUSTAKA

Jaringan Saraf Tiruan (ANN) sebagai klasifikasi data

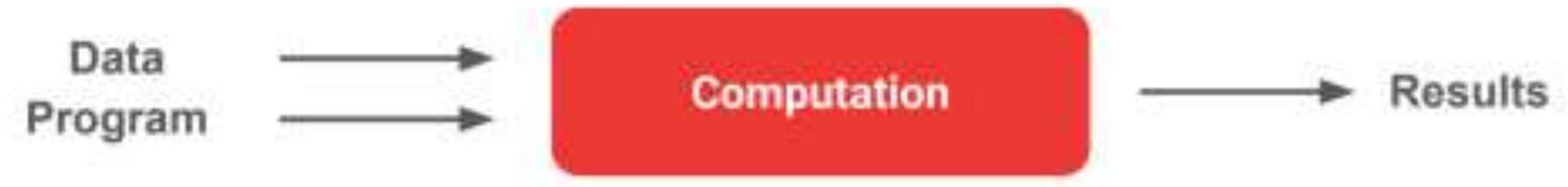


Condition of Comparison

Condition Evaluations

```
if hour < 12:  
    print("Good morning!")  
  
def check(number):  
    if number > 0:  
        return "Positive"  
    elif number == 0:  
        return "Zero"  
    else:  
        return "Negative"
```

Traditional Programming



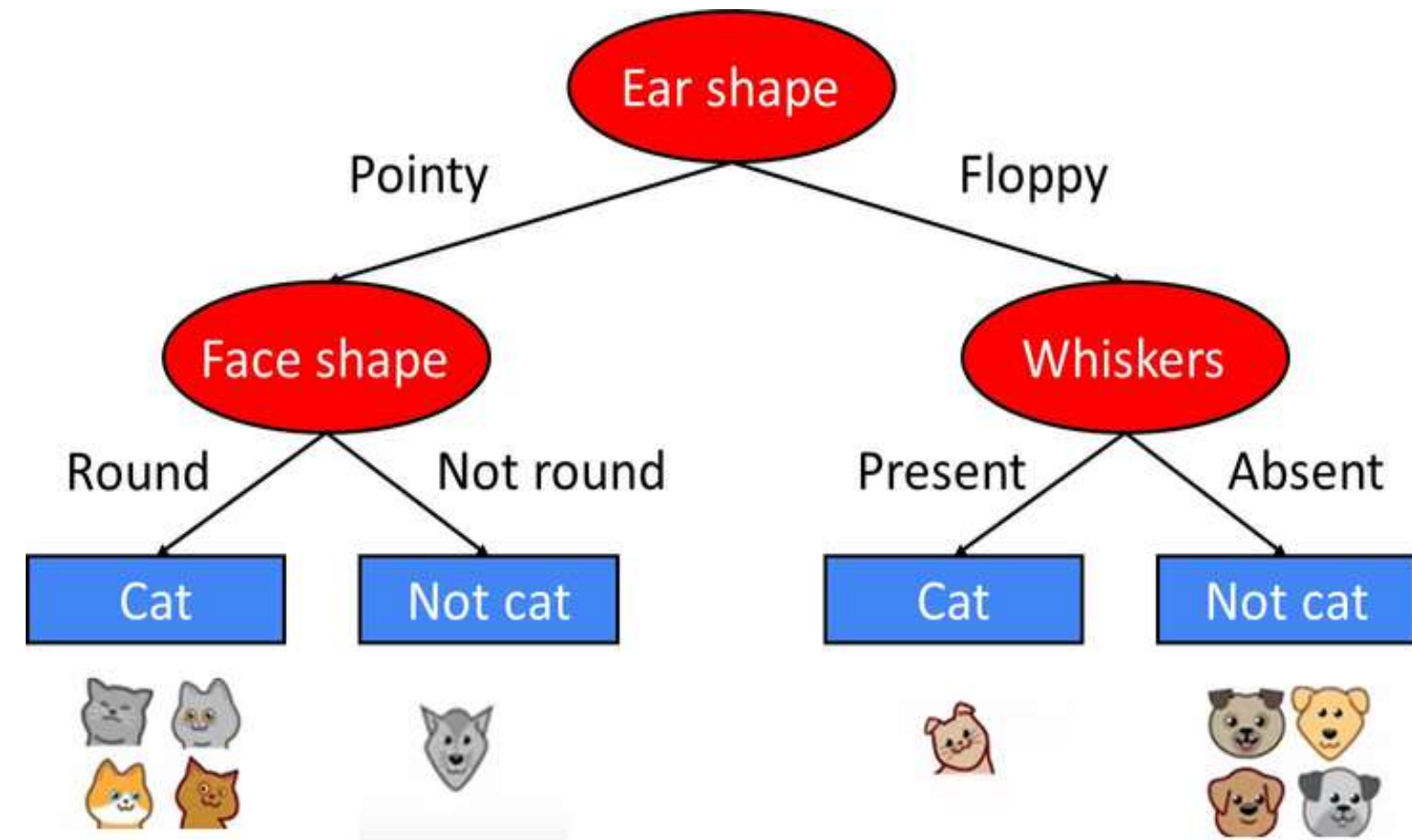
Bangkit Academy 2024 lead by Google and GoTo Batch 1 ILT-Tech 2 Materials, Machine Learning Path.



TINJAUAN PUSTAKA

Jaringan Saraf Tiruan (ANN) sebagai klasifikasi data

Ear shape	Face shape	Whiskers	Cat/No cat
Pointy	Round	Present	1
Floppy	Nor Round	Present	1
Floppy	Round	Absent	0
Pointy	Nor Round	Present	0
Pointy	Round	Present	1
Pointy	Round	Absent	1
Floppy	Nor Round	Absent	0
Pointy	Round	Absent	1
Floppy	Round	Absent	0
Floppy	Round	Absent	0



SKEMA ALAT

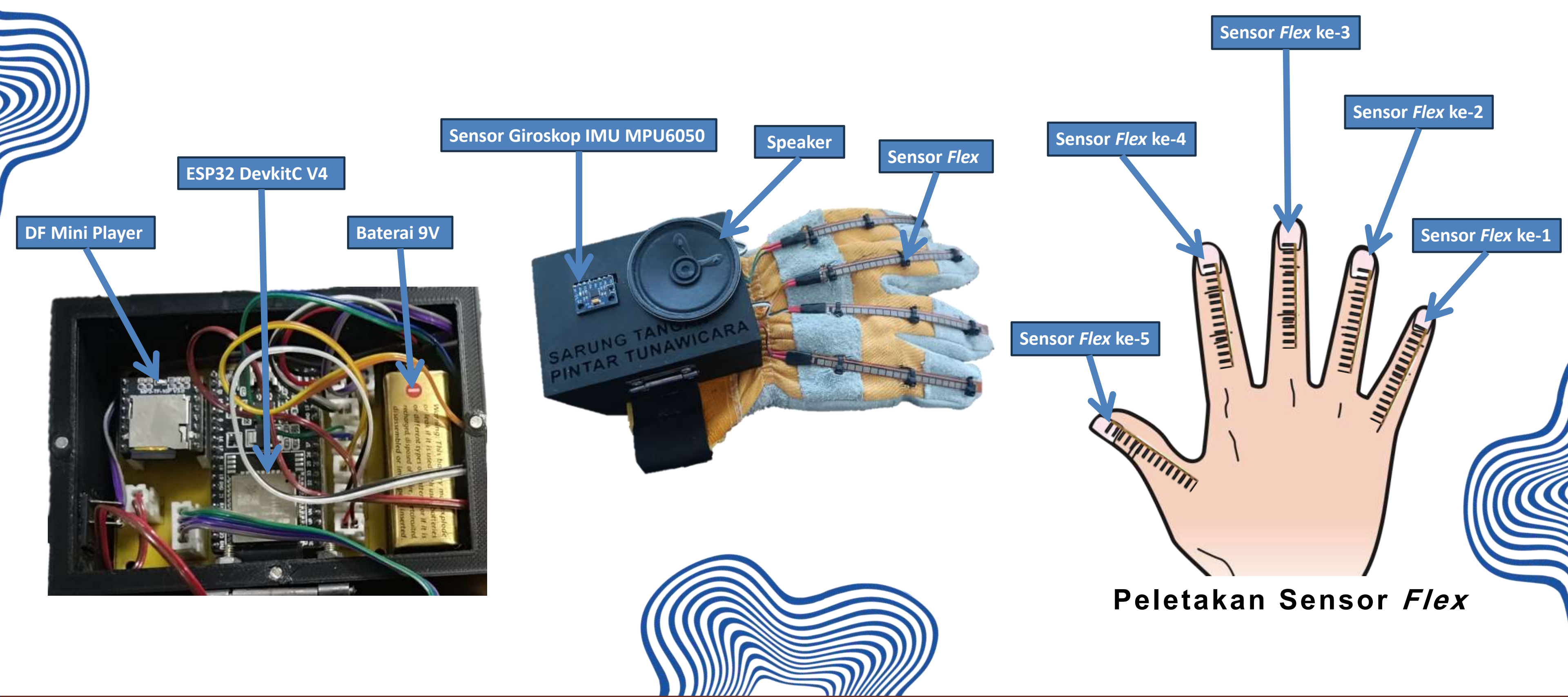


DIAGRAM BLOK RANGKAIAN

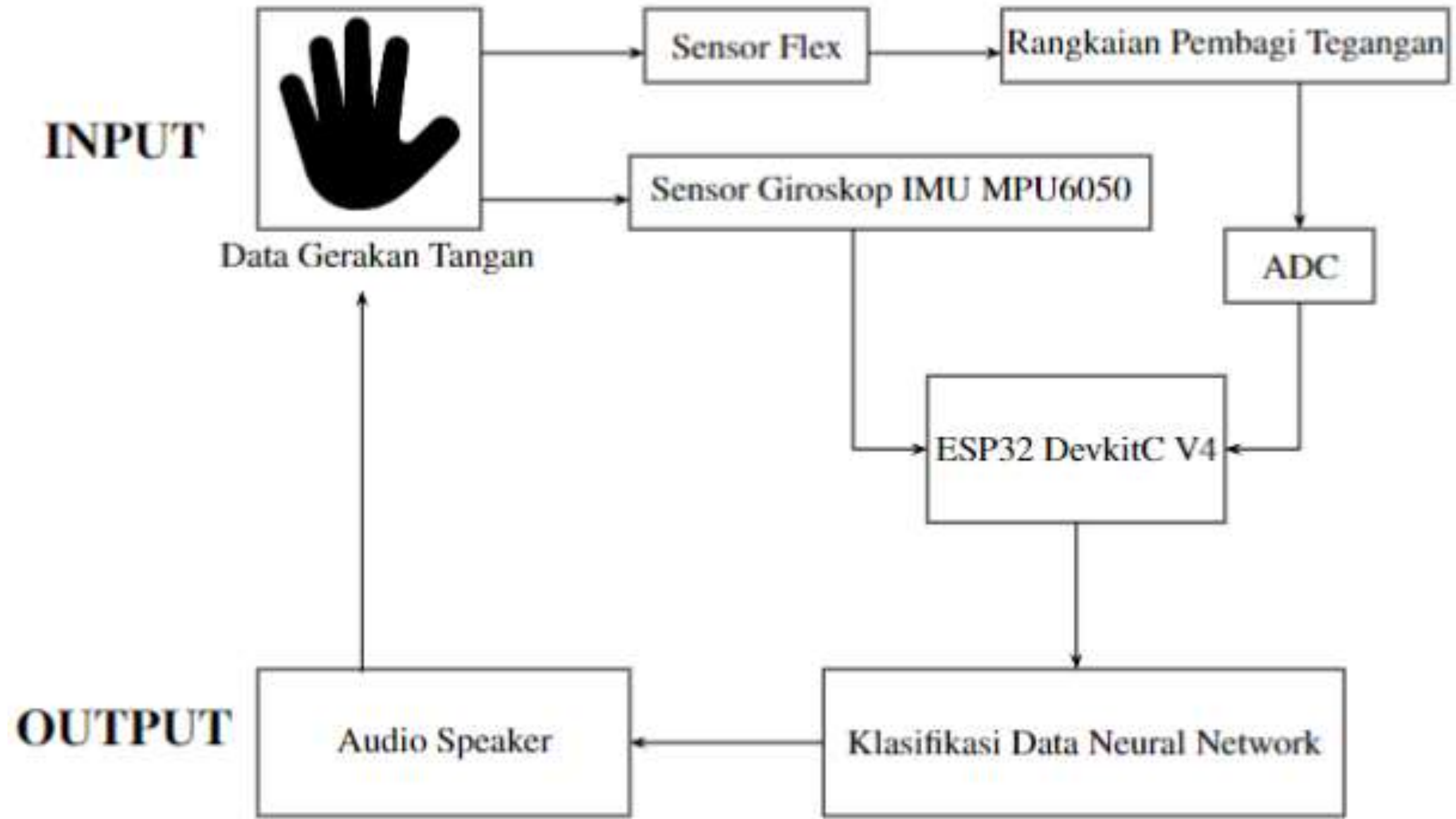
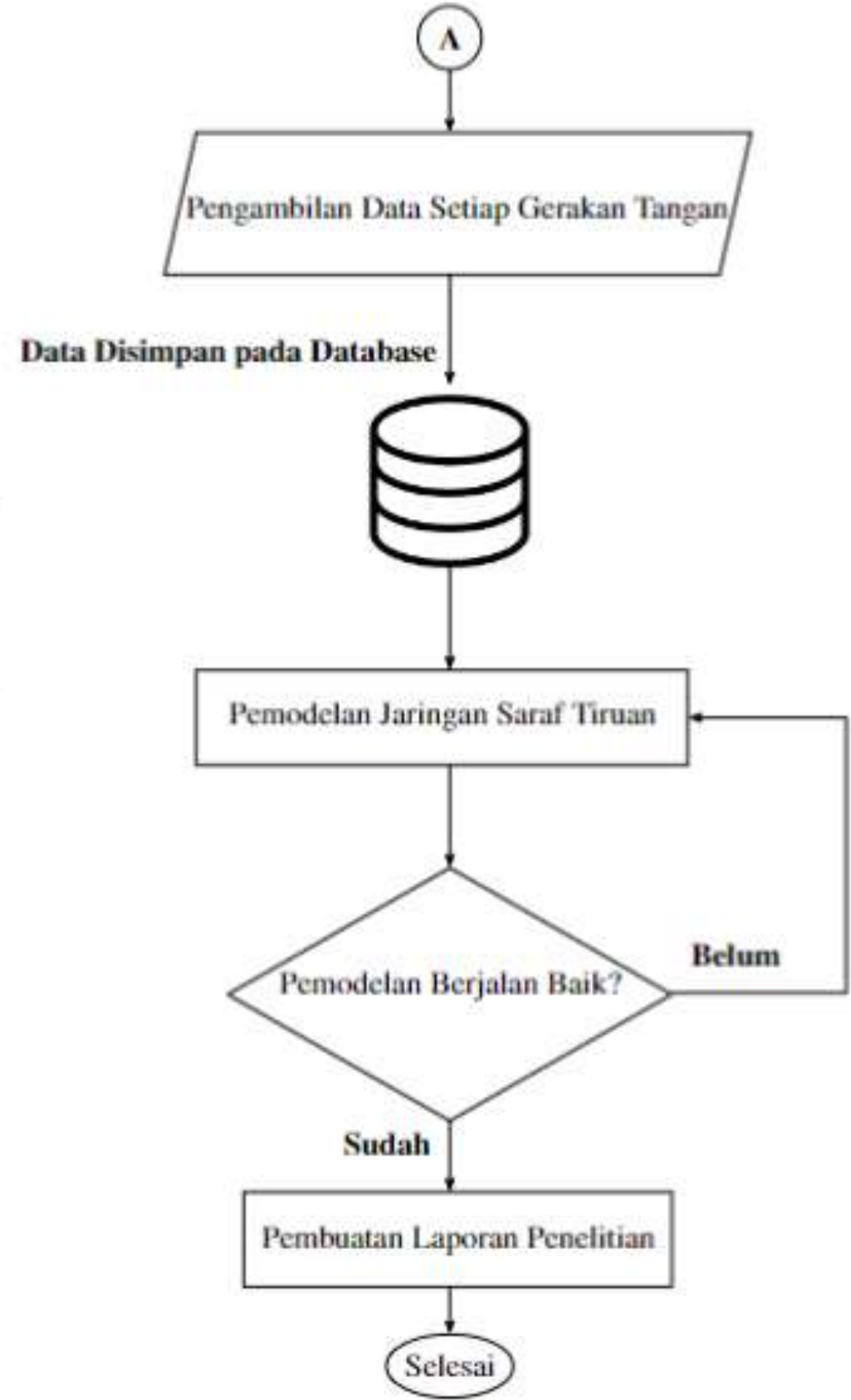
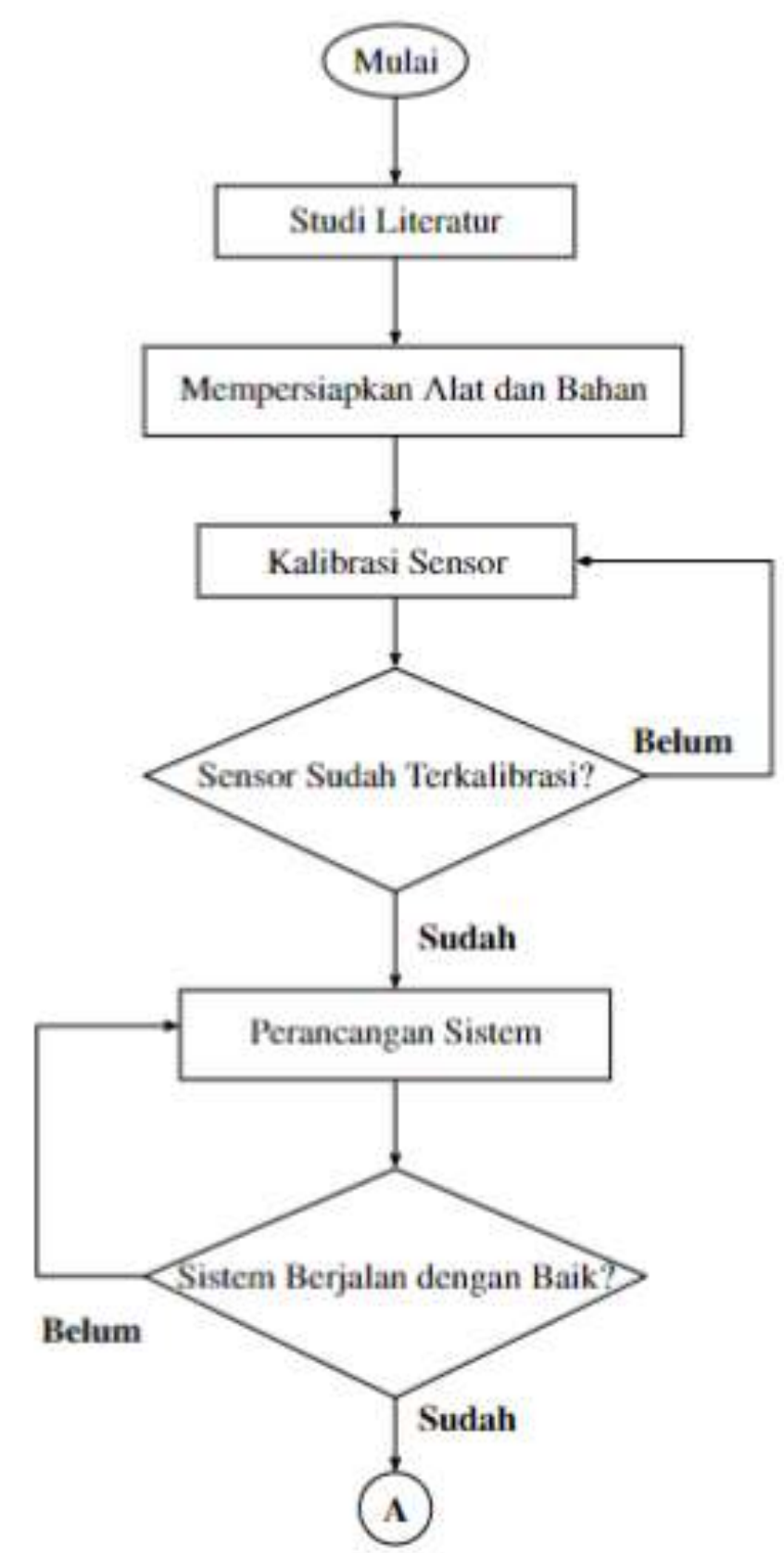


DIAGRAM ALIR PENELITIAN



METODOLOGI

Pengukuran Sensitivitas dan Kalibrasi Sensor

Pengukuran sensitivitas sensor *Flex* (resistantasi dan tegangan) dan kalibrasi Girooskop IMU MPU6050.

Pengambilan Data

Ekstraksi Data Gerakan pada setiap Abjad sesuai dengan Sistem Isyarat Bahasa Indonesia (SIBI) lalu dibuat pemodelan.

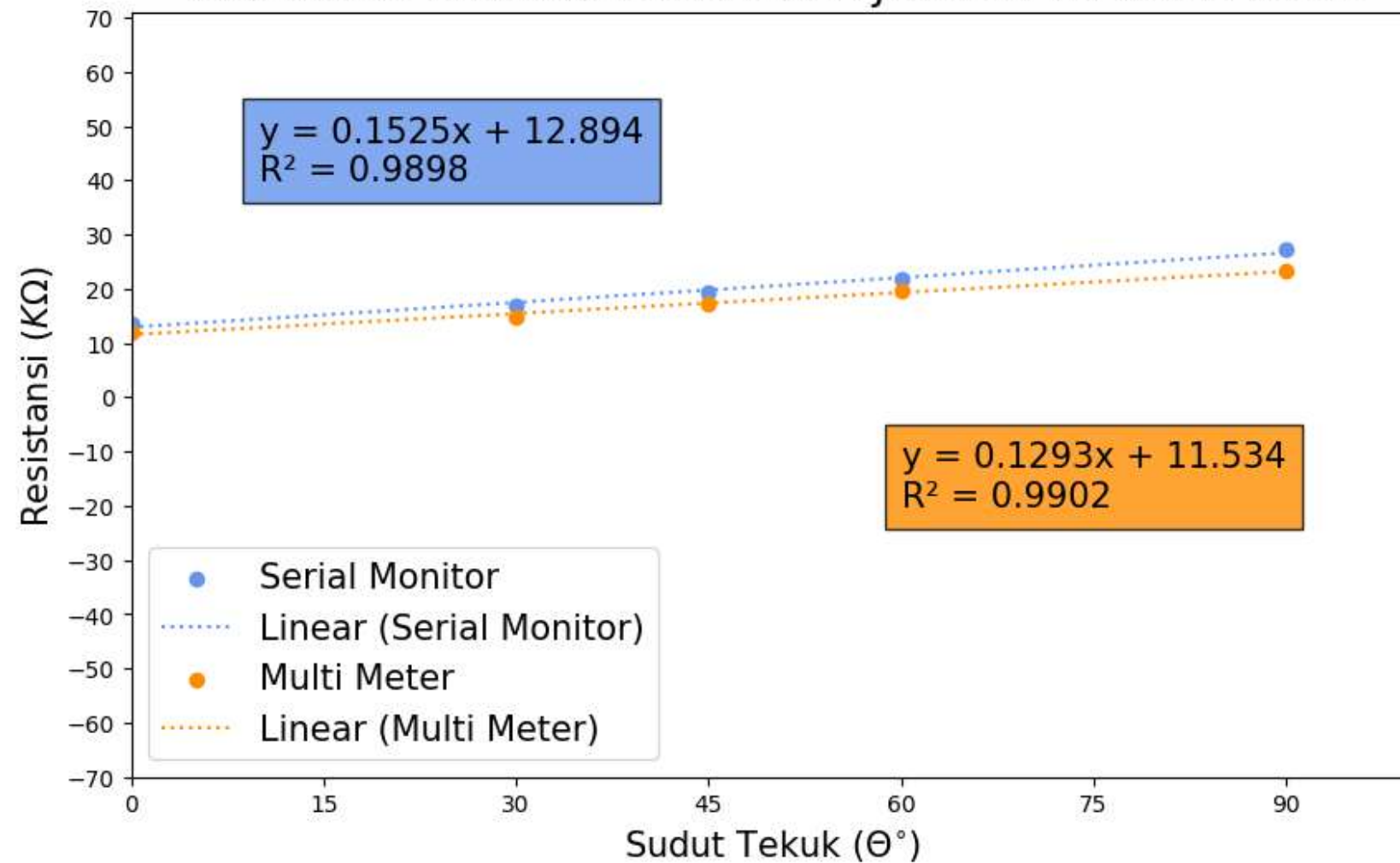
Evaluasi Pemodelan

Evaluasi model jaringan saraf tiruan (ANN) klasifikasi data gerakan Isyarat Bahasa Indonesia (SIBI) menggunakan *confusion matrix* dan grafik hasil pelatihan.

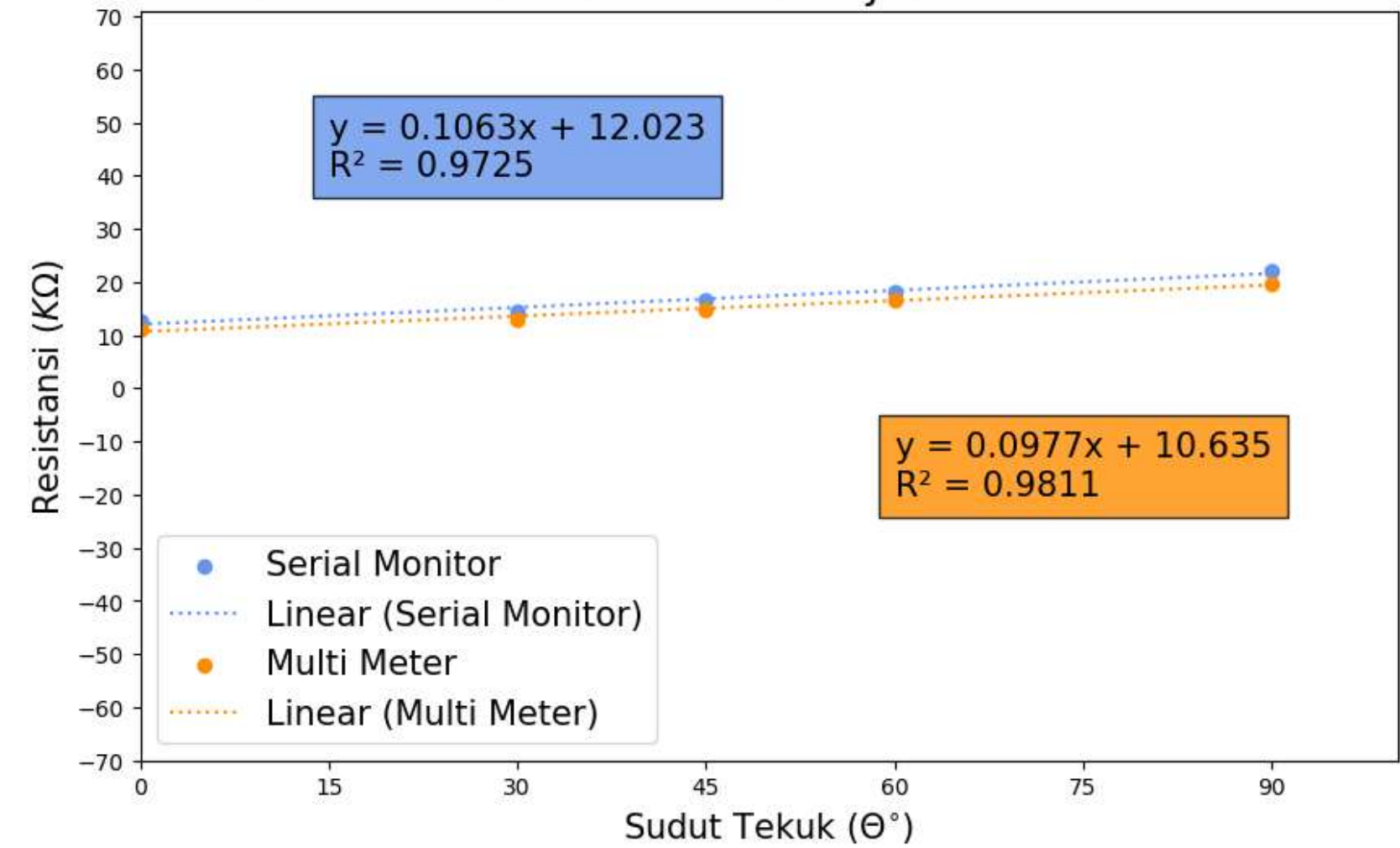
HASIL DAN PEMBAHASAN

Pengukuran Sensitivitas Sensor Flex : Resistansi terhadap sudut tekuk

Sensitivitas Resistansi Sensor *Flex* Jari Ke-1 VS Sudut Tekuk



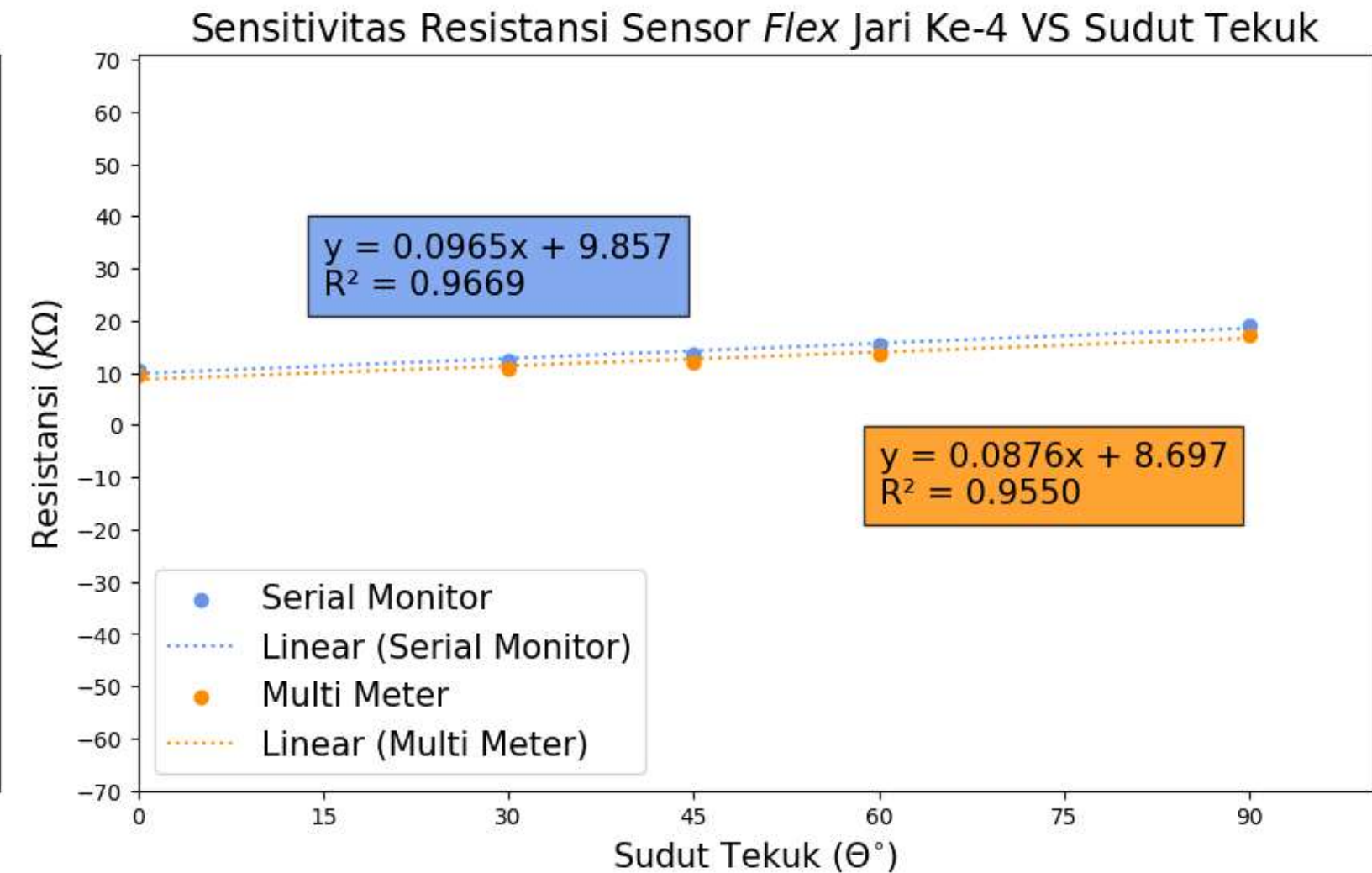
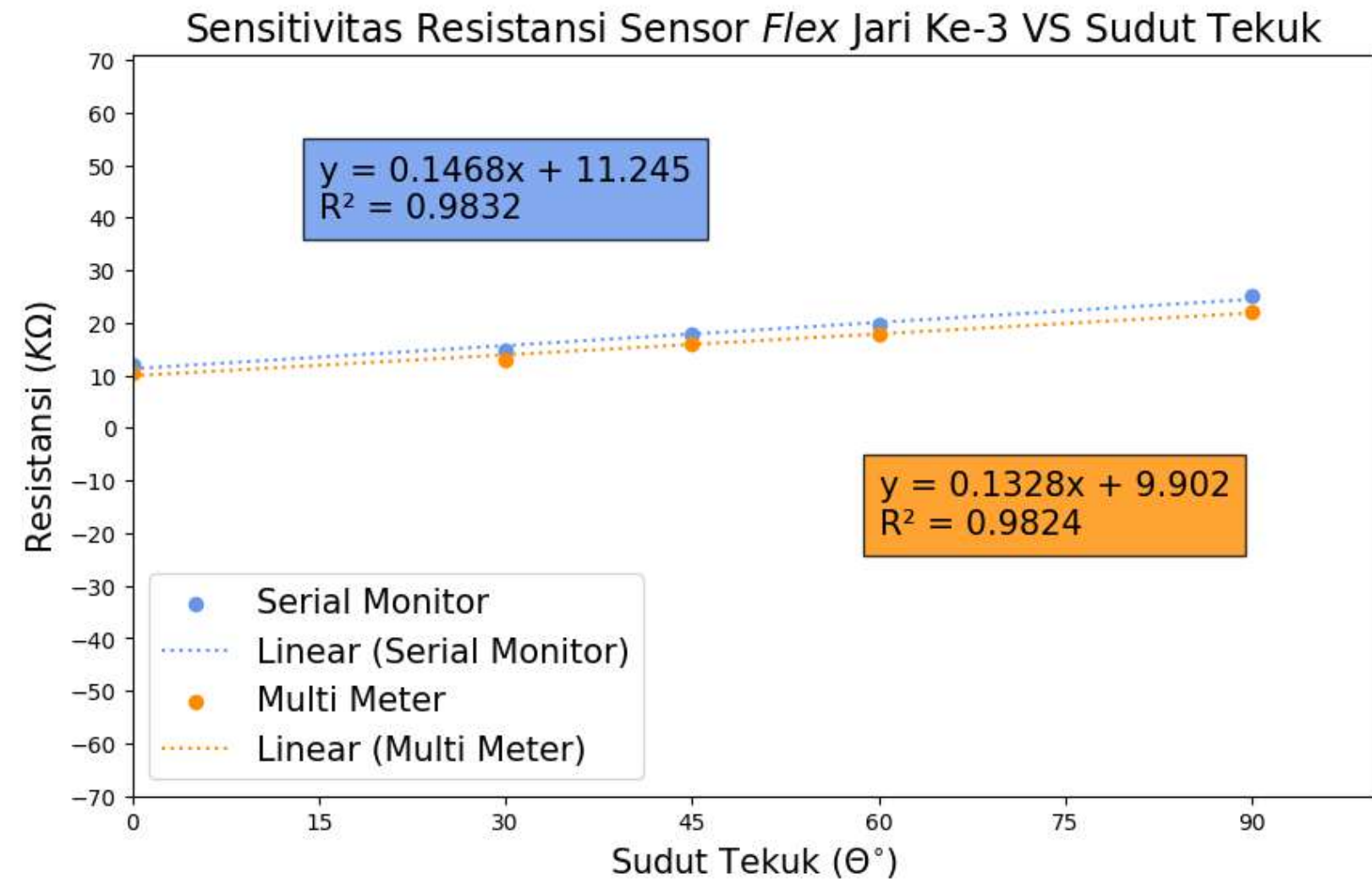
Sensitivitas Resistansi Sensor *Flex* Jari Ke-2 VS Sudut Tekuk



Sudut (θ°)	Sensor Flex ke-1 ($K\Omega$)		Sensor Flex ke-2 ($K\Omega$)	
	Multimeter	Serial Monitor	Multimeter	Serial Monitor
0	11,97	13,48	11,17	12,69
30	14,73	17,01	12,97	14,43
45	17,23	19,43	14,77	16,74
60	19,53	21,70	16,47	18,07
90	23,30	27,17	19,77	22,11

HASIL DAN PEMBAHASAN

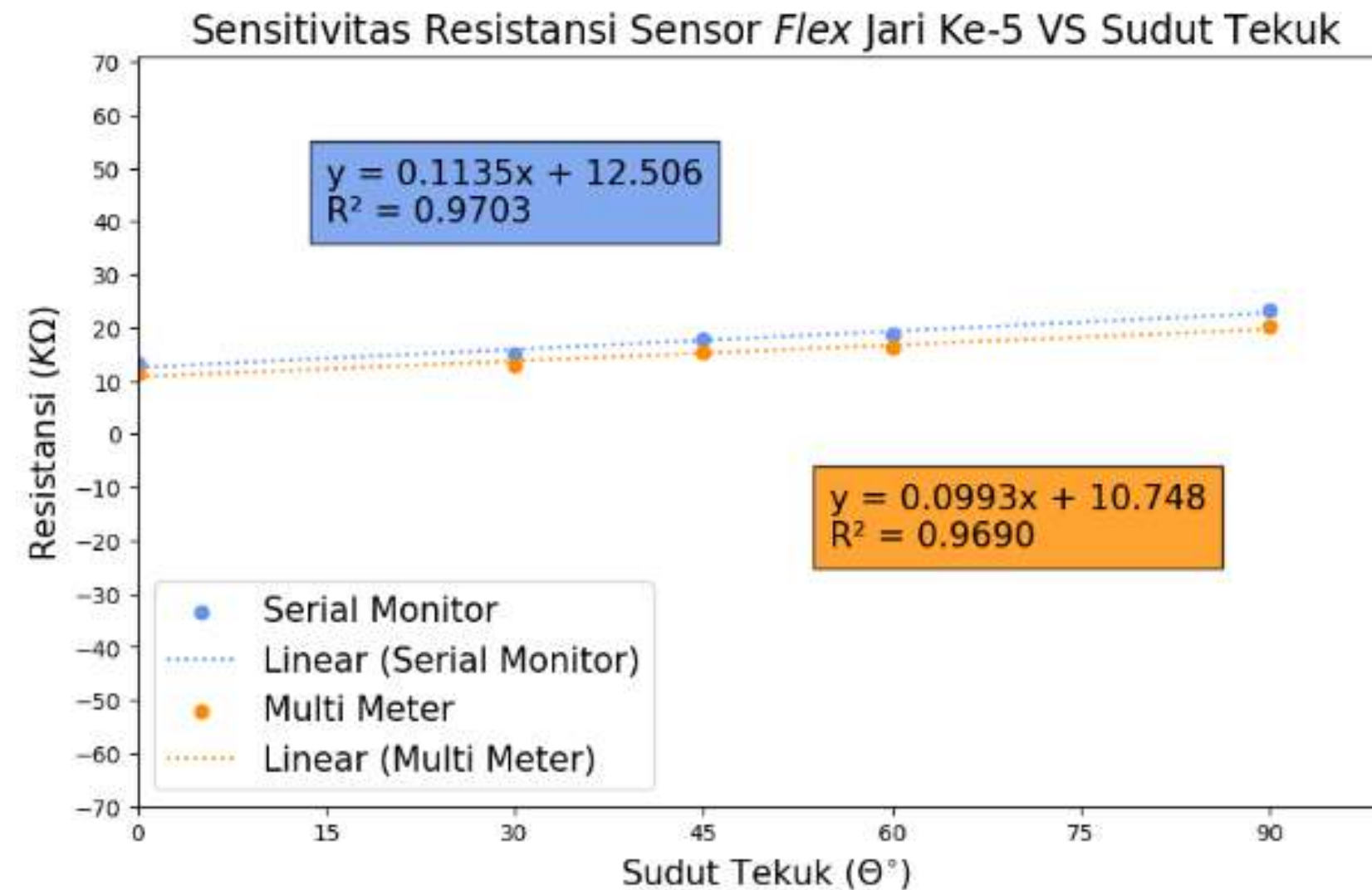
Pengukuran Sensitivitas Sensor Flex : Resistansi terhadap sudut tekuk



Sudut (θ°)	Sensor Flex ke-3 ($K\Omega$)		Sensor Flex ke-4 ($K\Omega$)	
	Multimeter	Serial Monitor	Multimeter	Serial Monitor
0	10,55	11,93	9,42	10,53
30	12,95	14,89	10,82	12,31
45	15,85	17,83	12,07	13,61
60	17,83	19,55	13,65	15,39
90	22,20	25,06	17,24	19,15

HASIL DAN PEMBAHASAN

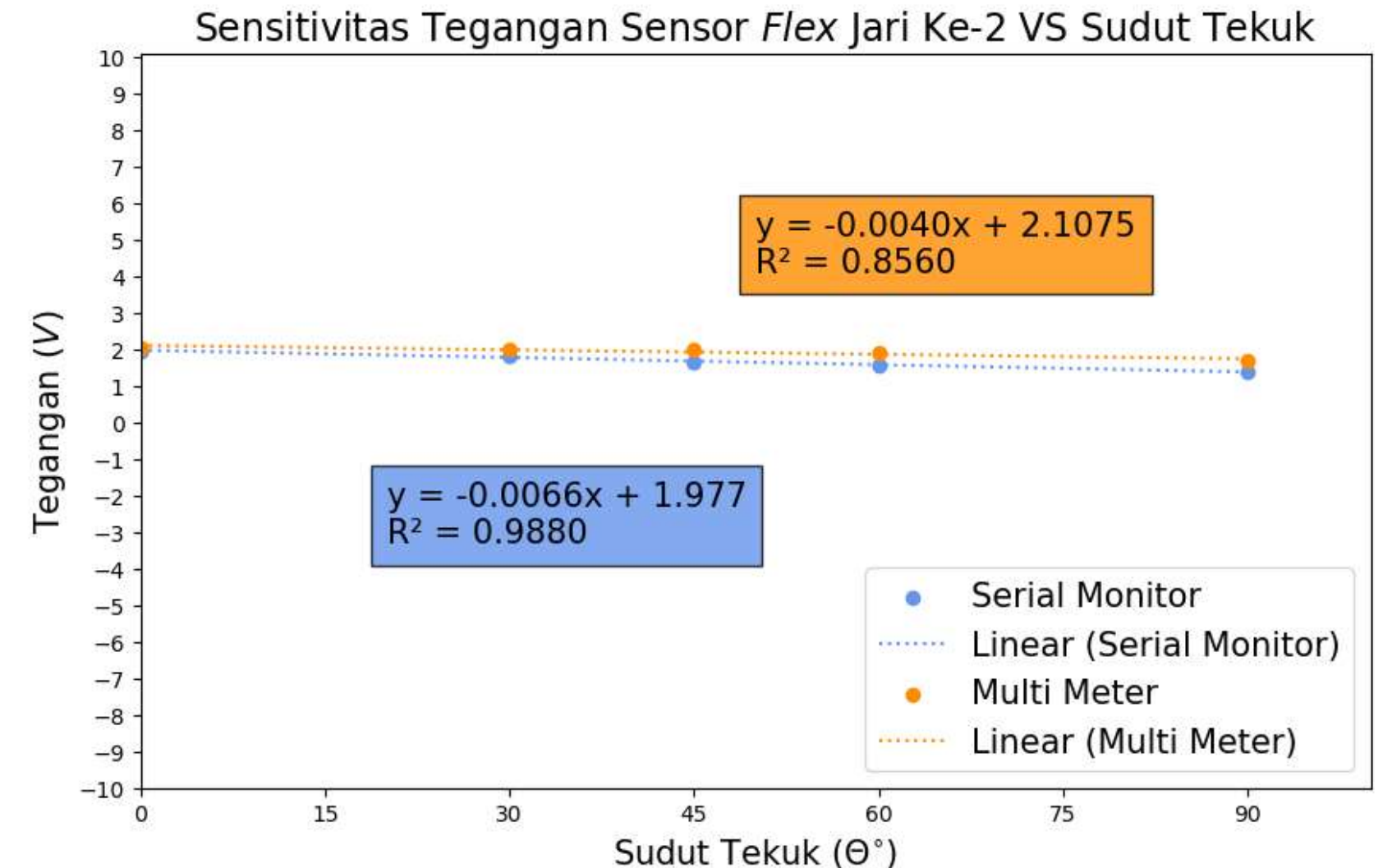
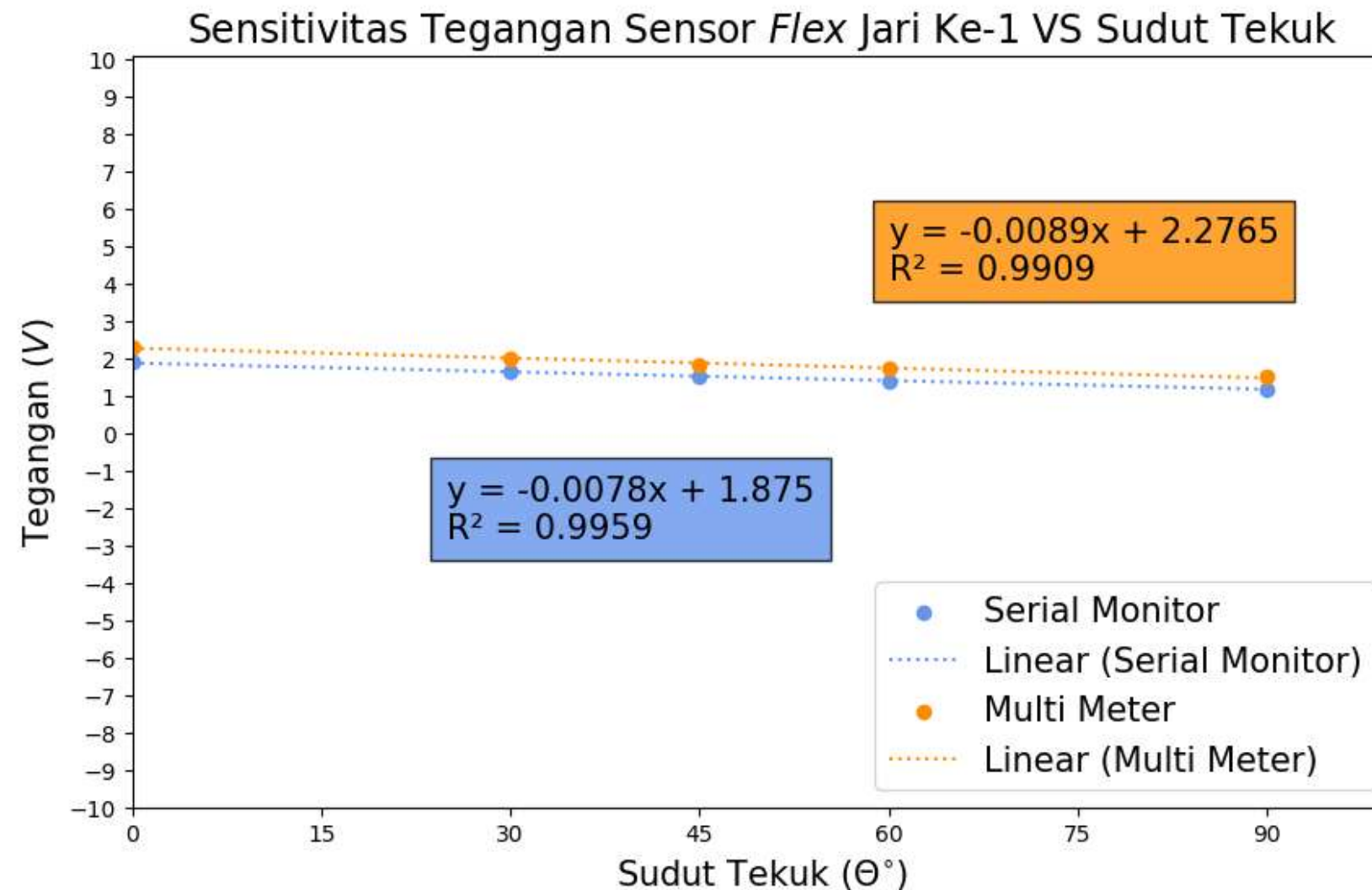
Pengukuran Sensitivitas Sensor Flex : Resistansi terhadap sudut tekuk



Sudut (θ°)	Sensor Flex ke-5 (KΩ)	
	Multimeter	Serial Monitor
0	11,37	13,14
30	12,97	15,21
45	15,27	17,74
60	16,27	18,63
90	20,20	23,35

HASIL DAN PEMBAHASAN

Pengukuran Sensitivitas Sensor Flex : Tegangan terhadap sudut tekuk

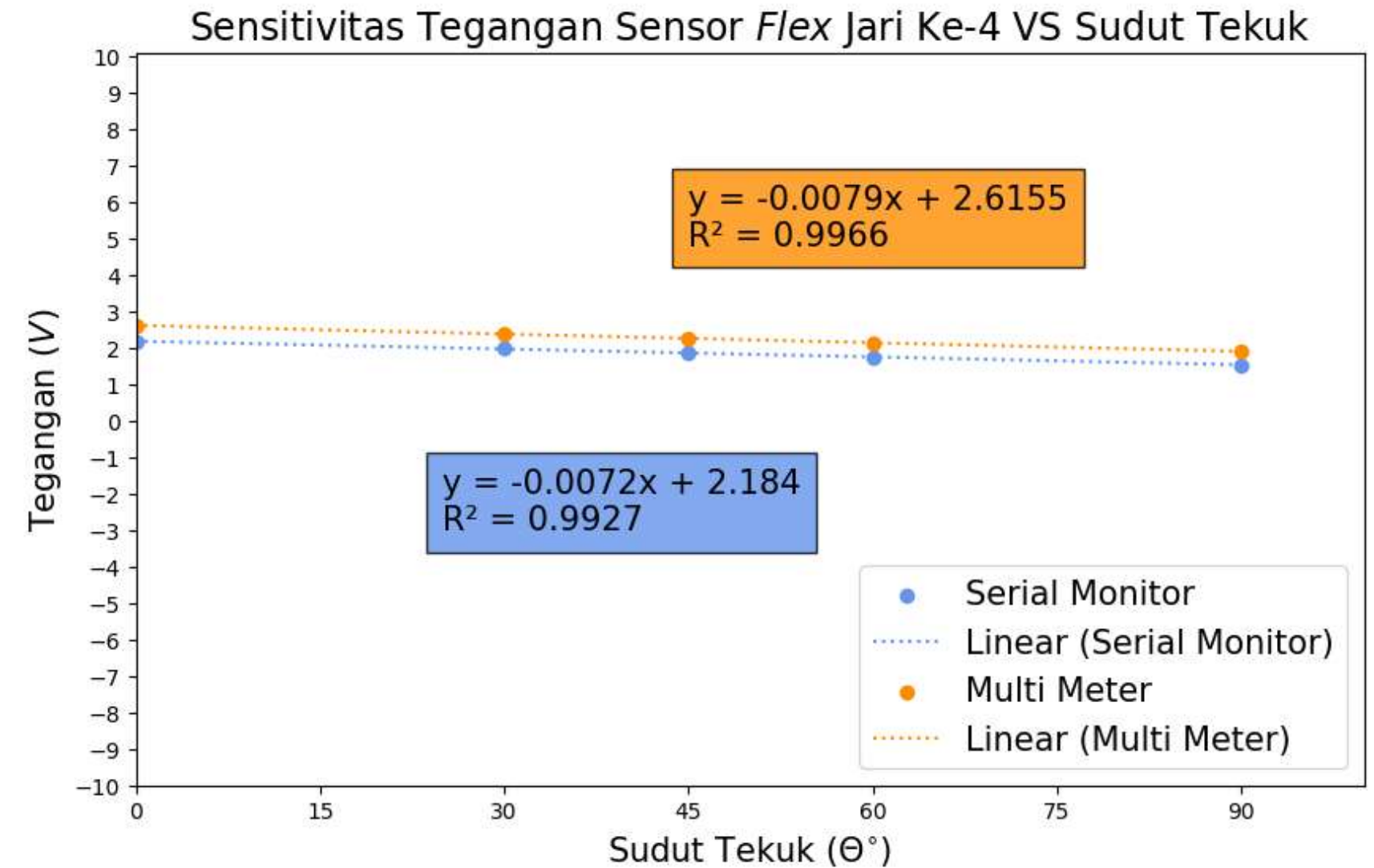
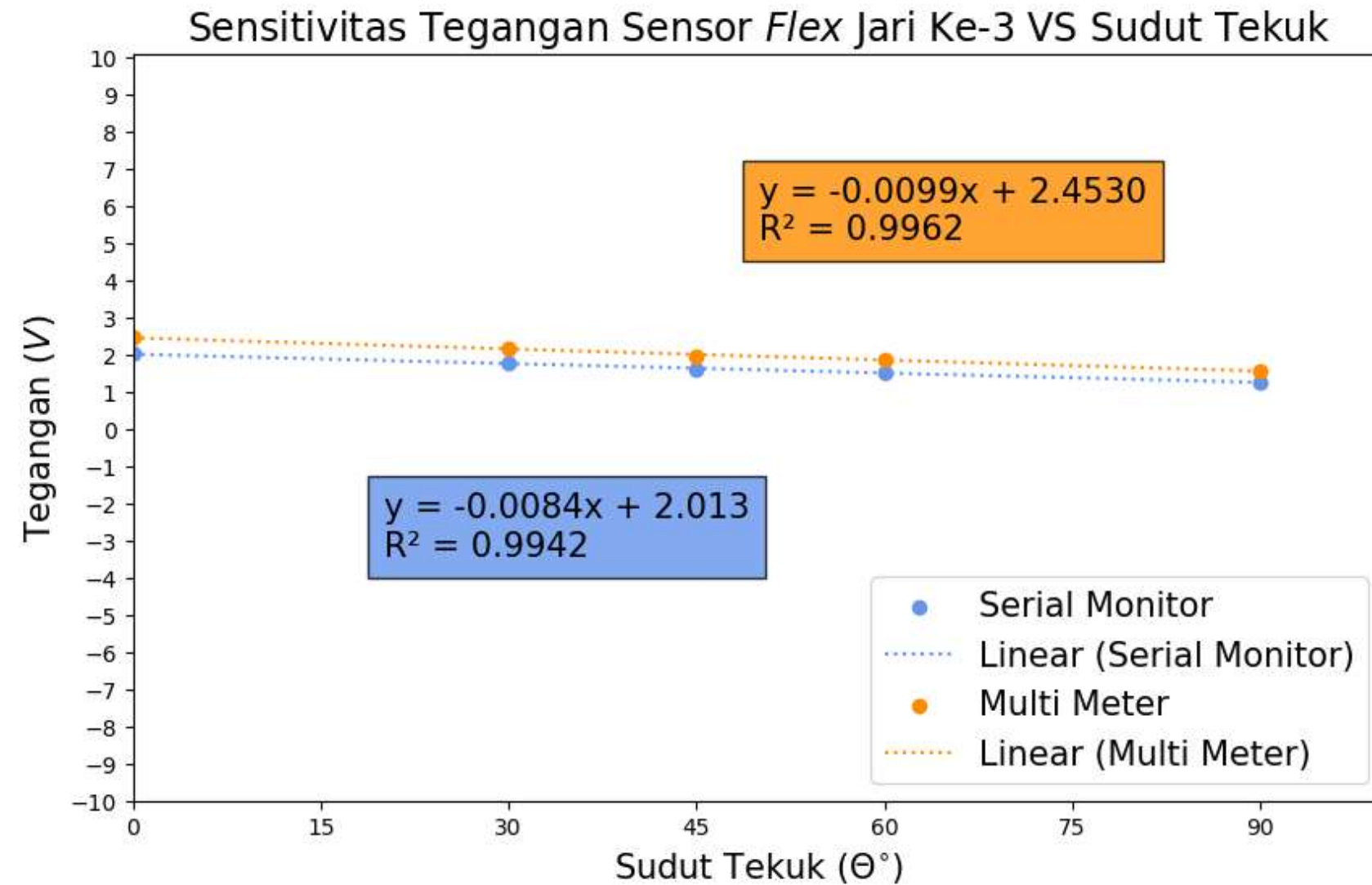


Sudut (θ°)	Sensor Flex ke-1 (V)		Sensor Flex ke-2 (V)	
	Multimeter	Serial Monitor	Multimeter	Serial Monitor
0	2,3	1,89	2,06	1,96
30	2,01	1,64	1,99	1,82
45	1,83	1,5	1,98	1,66
60	1,74	1,4	1,92	1,58
90	1,5	1,19	1,68	1,38

HASIL DAN PEMBAHASAN



Pengukuran Sensitivitas Sensor Flex : Tegangan terhadap sudut tekuk

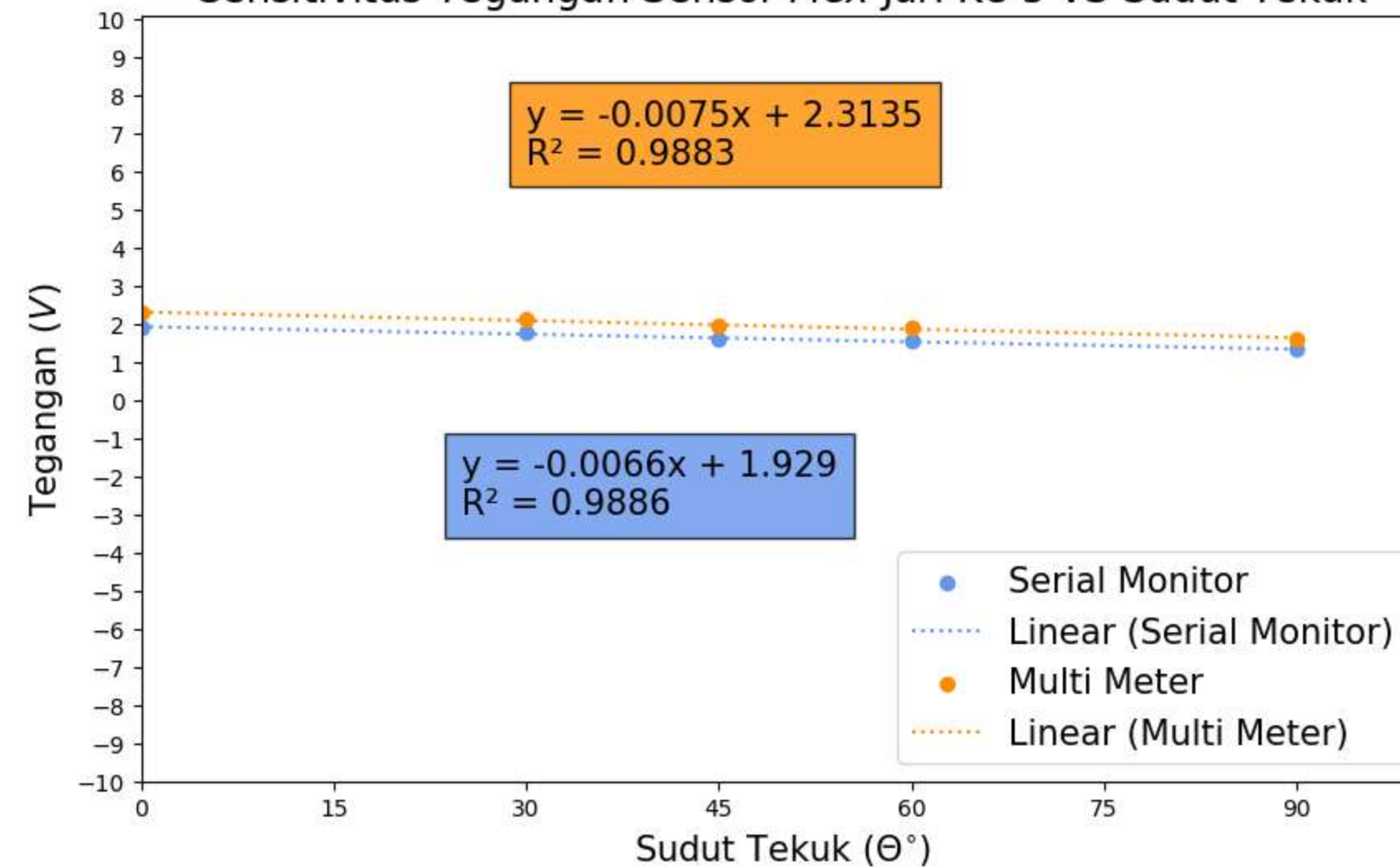


Sudut (θ°)	Sensor Flex ke-3 (V)		Sensor Flex ke-4 (V)	
	Multimeter	Serial Monitor	Multimeter	Serial Monitor
0	2,46	2,02	2,6	2,16
30	2,17	1,78	2,4	1,99
45	1,97	1,6	2,27	1,88
60	1,86	1,5	2,13	1,75
90	1,57	1,27	1,9	1,52

HASIL DAN PEMBAHASAN

Pengukuran Sensitivitas Sensor Flex : Tegangan terhadap sudut tekuk

Sensitivitas Tegangan Sensor *Flex* Jari Ke-5 VS Sudut Tekuk



Sudut (θ°)	Sensor Flex ke-5 (V)	
	Multimeter	Serial Monitor
0	2,3	1,92
30	2,11	1,76
45	1,95	1,6
60	1,9	1,55
90	1,62	1,33

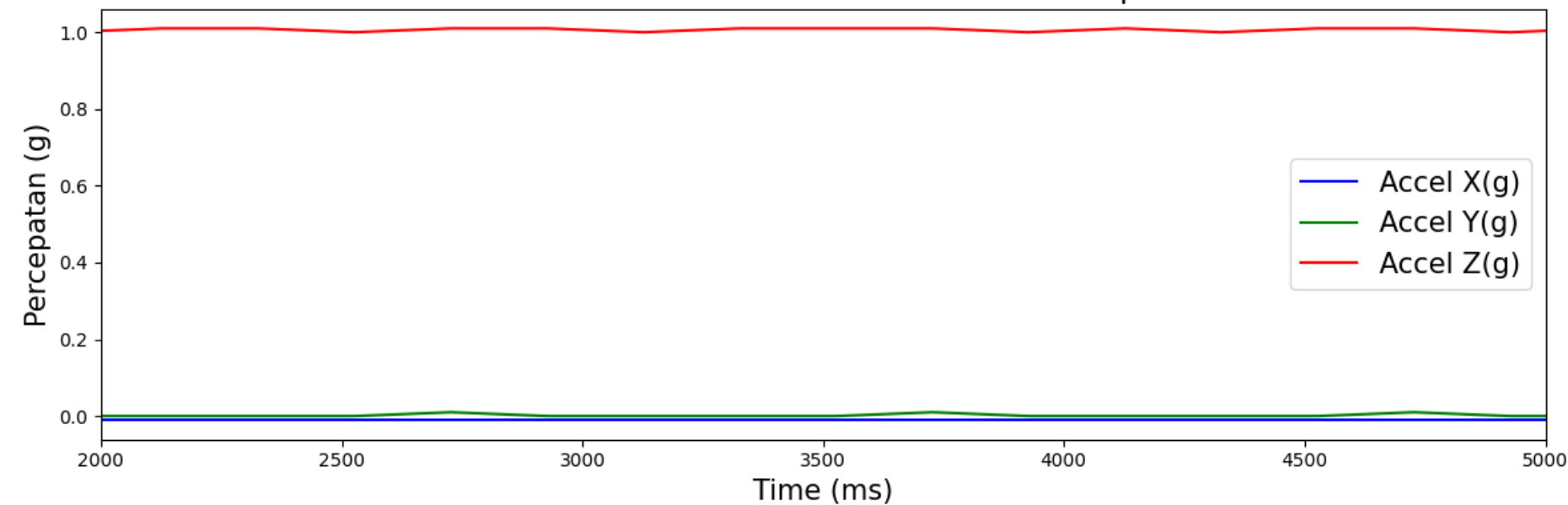
TINJAUAN PUSTAKA

Pengukuran Kalibrasi Sensor Giroskop IMU MPU6050

Data Accelerometer Sebelum Kalibrasi Terhadap Waktu



Data Accelerometer Setelah Kalibrasi Terhadap Waktu

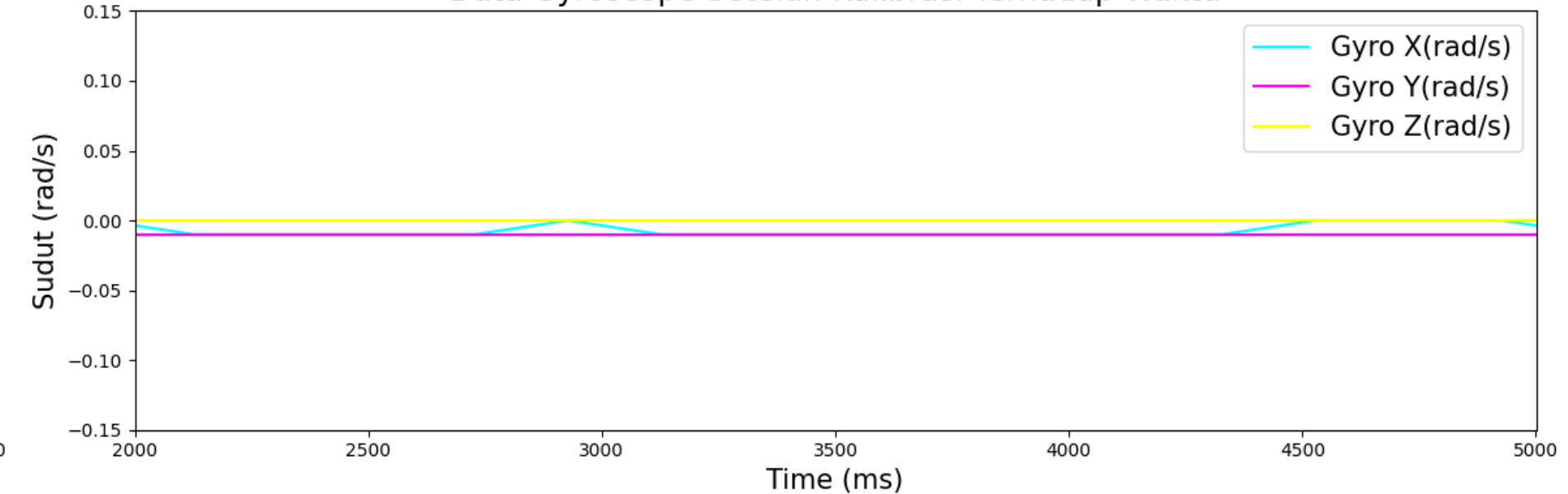


Data Percepatan Sudut ($1g = 9,81 \text{ m/s}^2$)

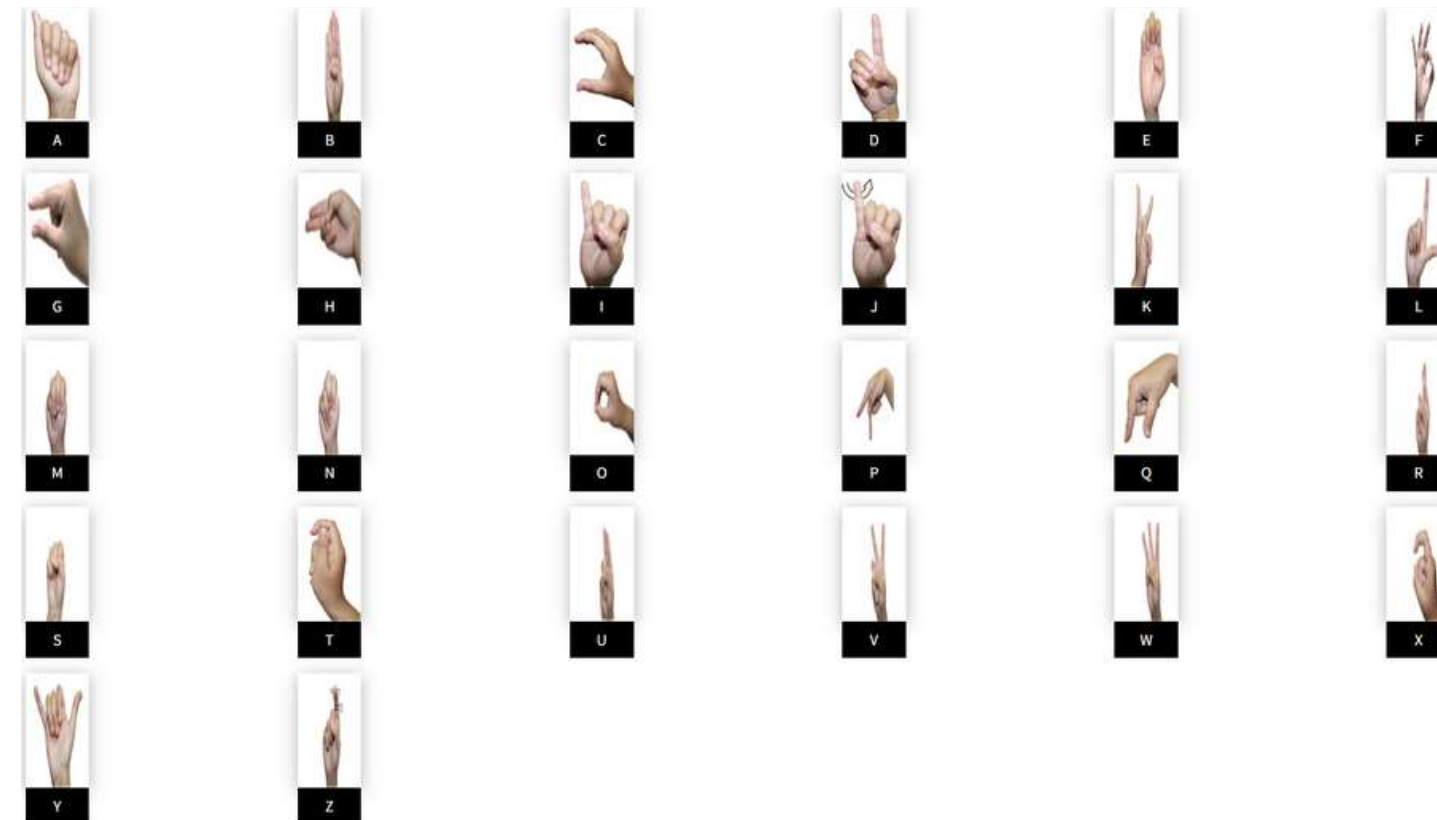
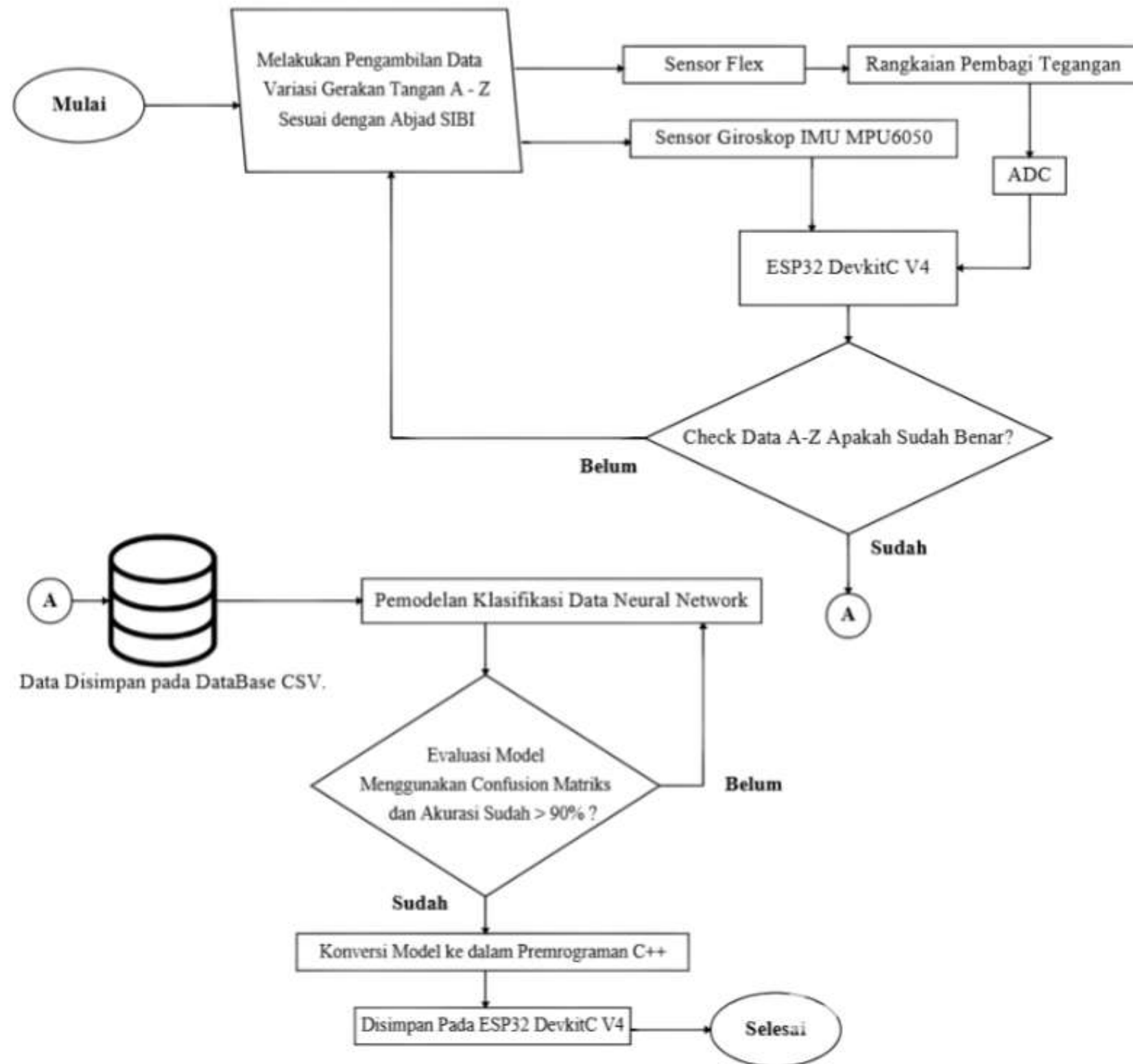
Data Gyroscope Sebelum Kalibrasi Terhadap Waktu



Data Gyroscope Setelah Kalibrasi Terhadap Waktu



Data Sudut Rotasi (rad/s)



Pengambilan data sesuai dengan gerakan yang berlaku pada aturan Sistem Isyarat Bahasa Indonesia yang menggunakan satu tangan, melafalkan abjad dari huruf A - Z .

Diagram Alir Pengambilan dan Pemrosesan Data

PENGAMBIILAN DATA

Tabel 5.6. Data Huruf A (ClassID = 0) Sarung Tangan Pintar.

No	Flex Ke-5	Flex Ke-4	Flex Ke-3	Flex Ke-2	Flex Ke-1	Accel-X	Accel-Y	Accel-Z	Gyro-X	Gyro-Y	Gyro-Z	ClassID
1	1421	1200	1114	1109	818	0.15	0.99	0.12	0.03	-0.04	0.01	0
2	1431	1207	1109	1118	805	0.16	1	0.15	-0.01	0.02	0.01	0
3	1456	1232	1118	1113	844	0.17	0.98	0.14	-0.03	-0.03	0	0
4	1417	1269	1079	1136	816	0.14	0.99	0.14	0	-0.04	-0.01	0
---	---	---	---	---	---	---	---	---	---	---	---	---
400	1447	1226	1123	1136	848	0.16	0.95	0.26	-0.06	0.02	0.01	0

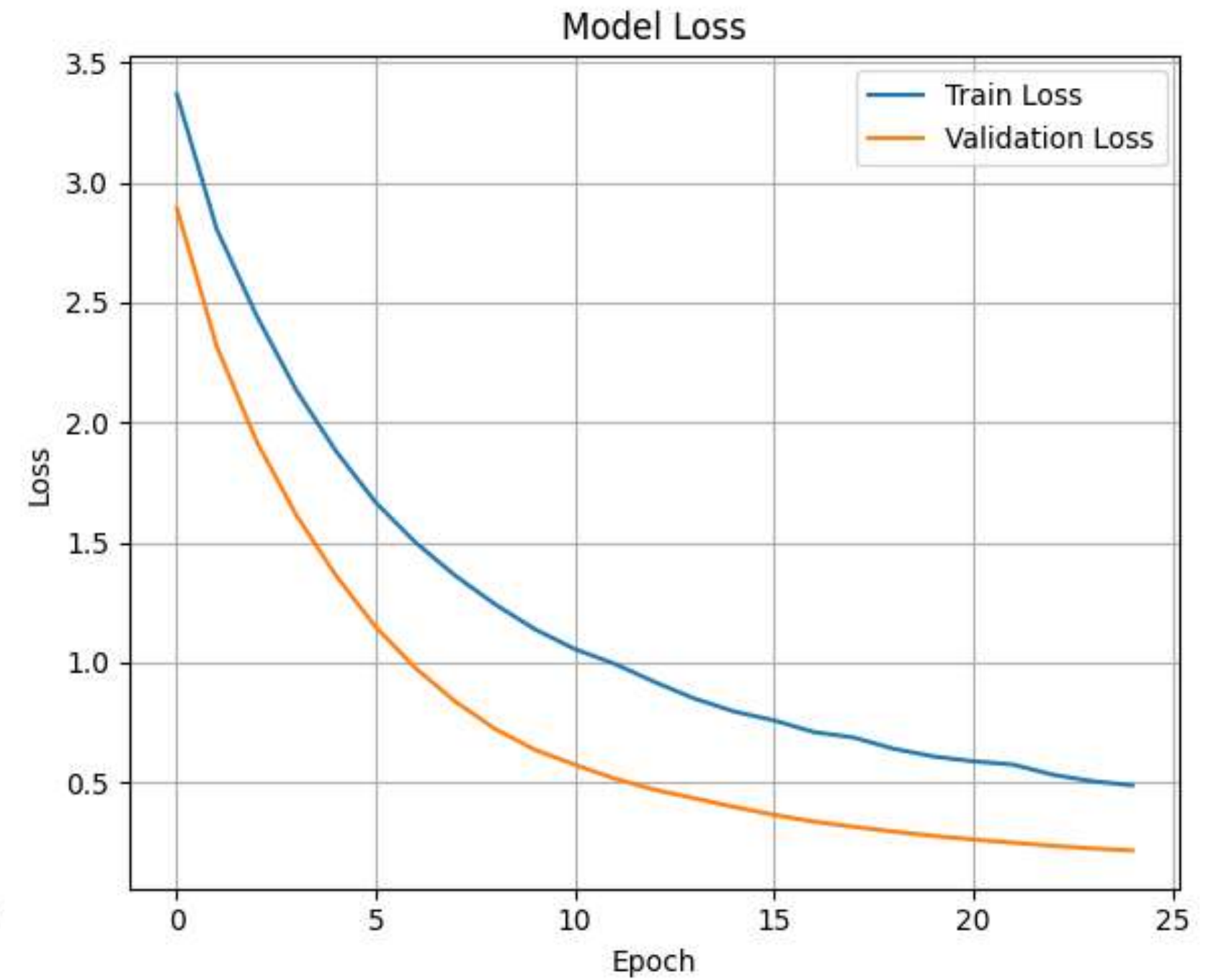
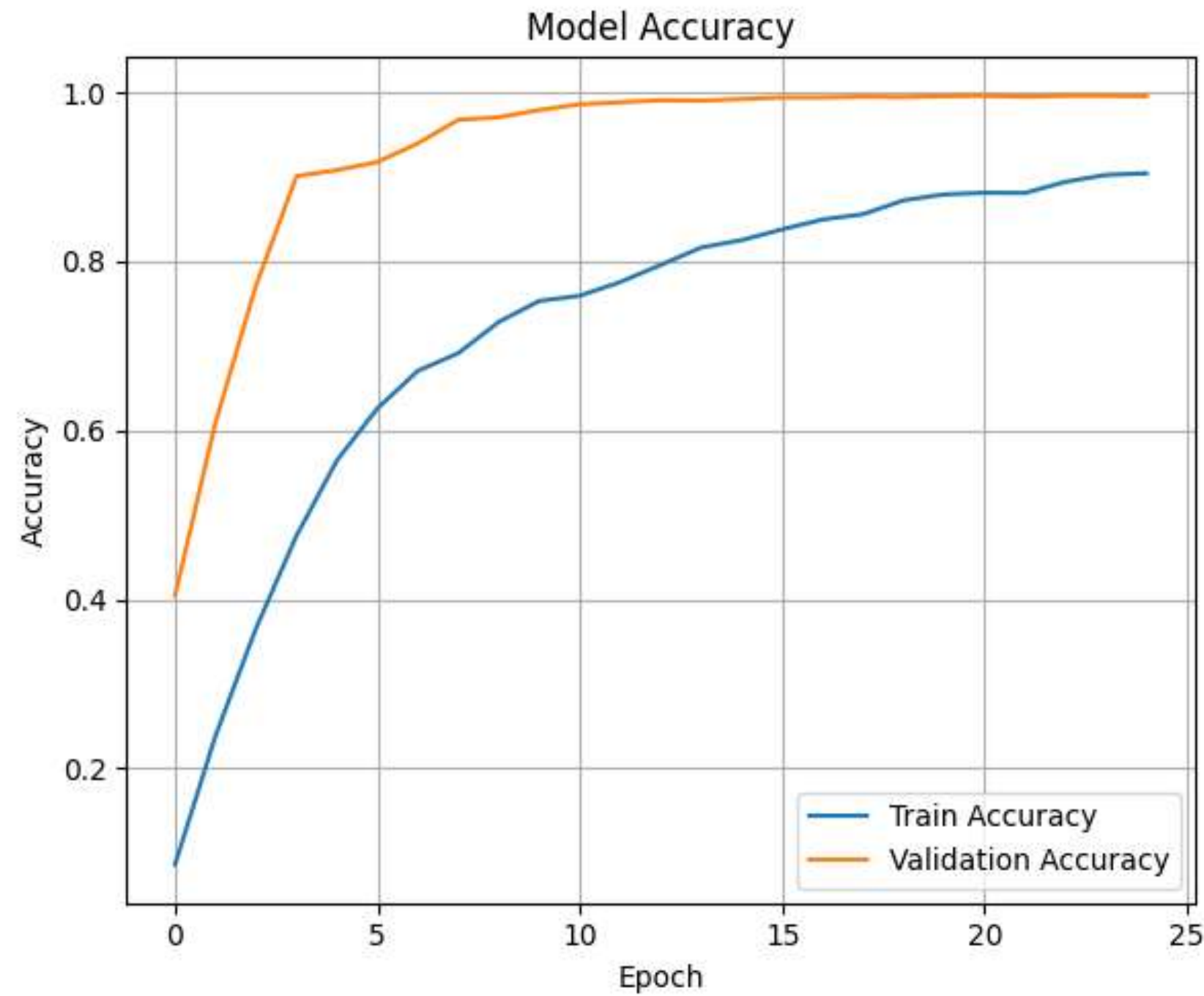
Tabel 5.7. Data Huruf B (ClassID = 1) Sarung Tangan Pintar.

No	Flex Ke-5	Flex Ke-4	Flex Ke-3	Flex Ke-2	Flex Ke-1	Accel-X	Accel-Y	Accel-Z	Gyro-X	Gyro-Y	Gyro-Z	ClassID
1	1200	1904	1766	1728	1535	0.04	0.19	0	0	0.02	-0.02	1
2	1203	1908	1763	1719	1535	0.21	0.98	0.16	0.01	0.04	-0.01	1
3	1263	1914	1764	1717	1562	0.23	0.98	0.15	0.01	0.03	0	1
4	1217	1911	1763	1729	1564	0.21	0.98	0.17	-0.01	0.05	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---
400	1264	1901	1776	1712	1595	0.21	0.97	0.21	-0.02	-0.03	0.01	1

Salah satu contoh data yang akan diolah pada pemodelan jaringan syaraf buatan, nantinya data akan dibagi menjadi 80% data pelatihan (*train*) dan 20% data validasi (*test*)



HASIL PEMODELAN JARINGAN SARAF TIRUAN

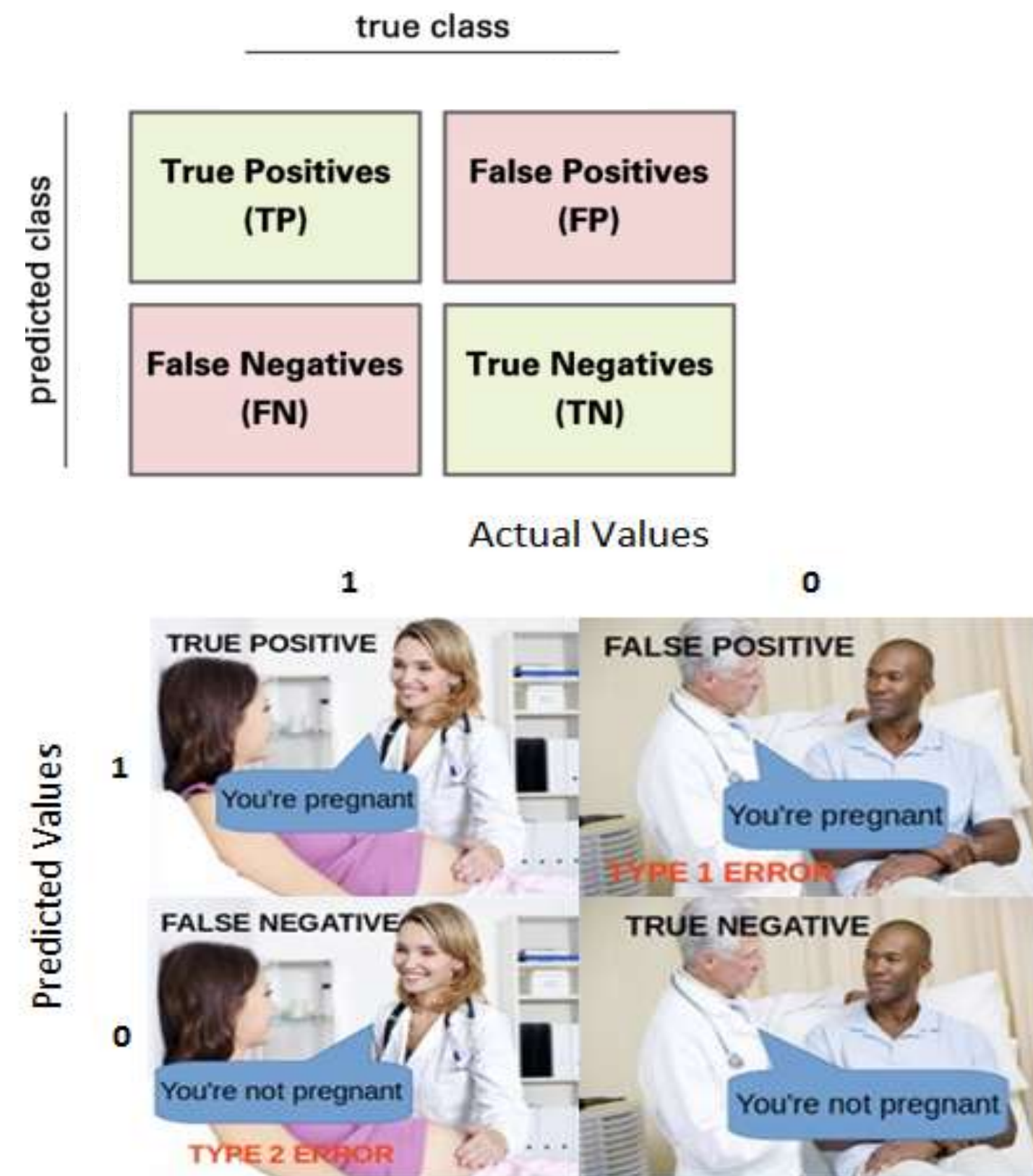
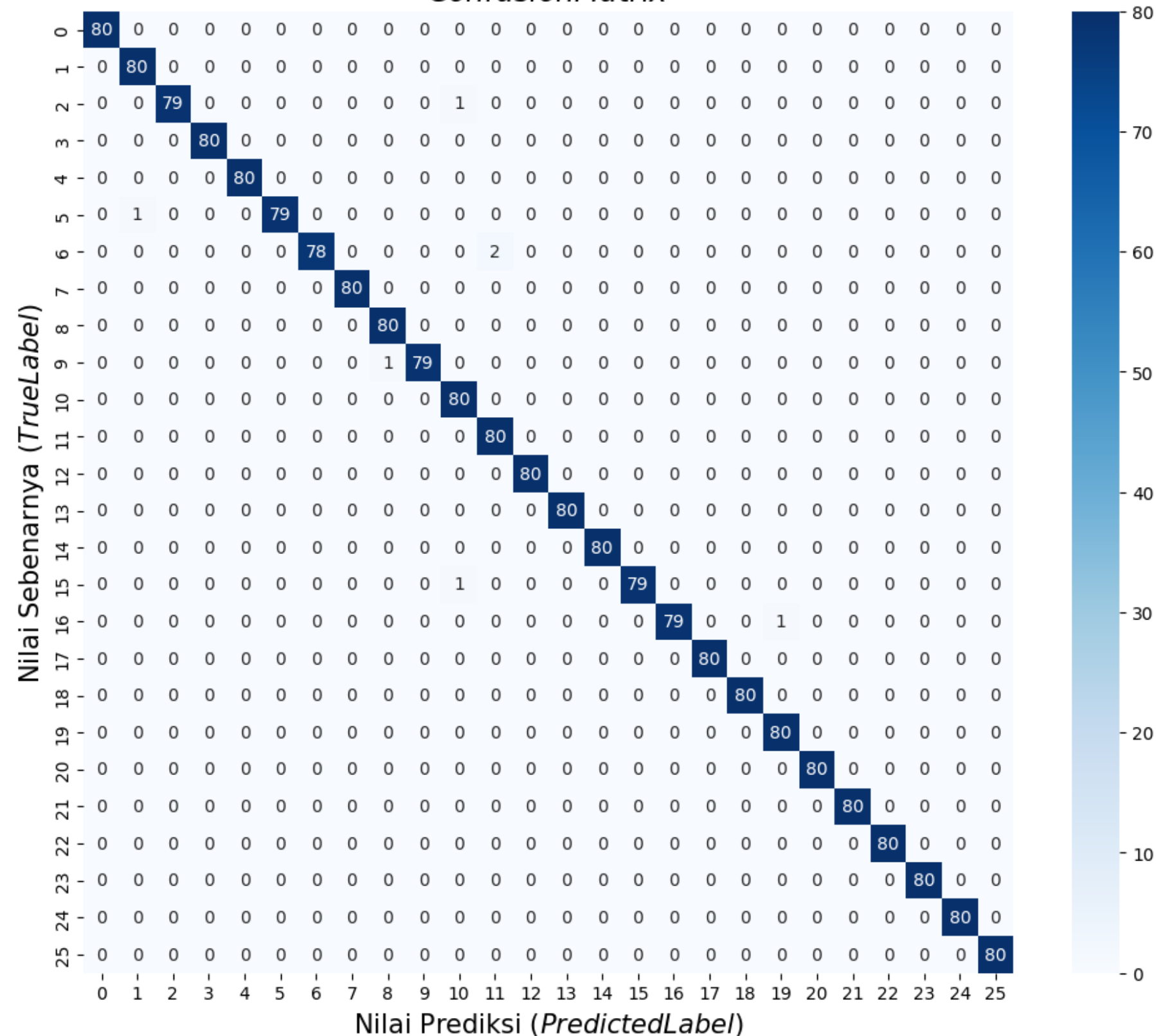


65/65 ————— 1s 3ms/step - accuracy: 0.9983 - loss: 0.2595

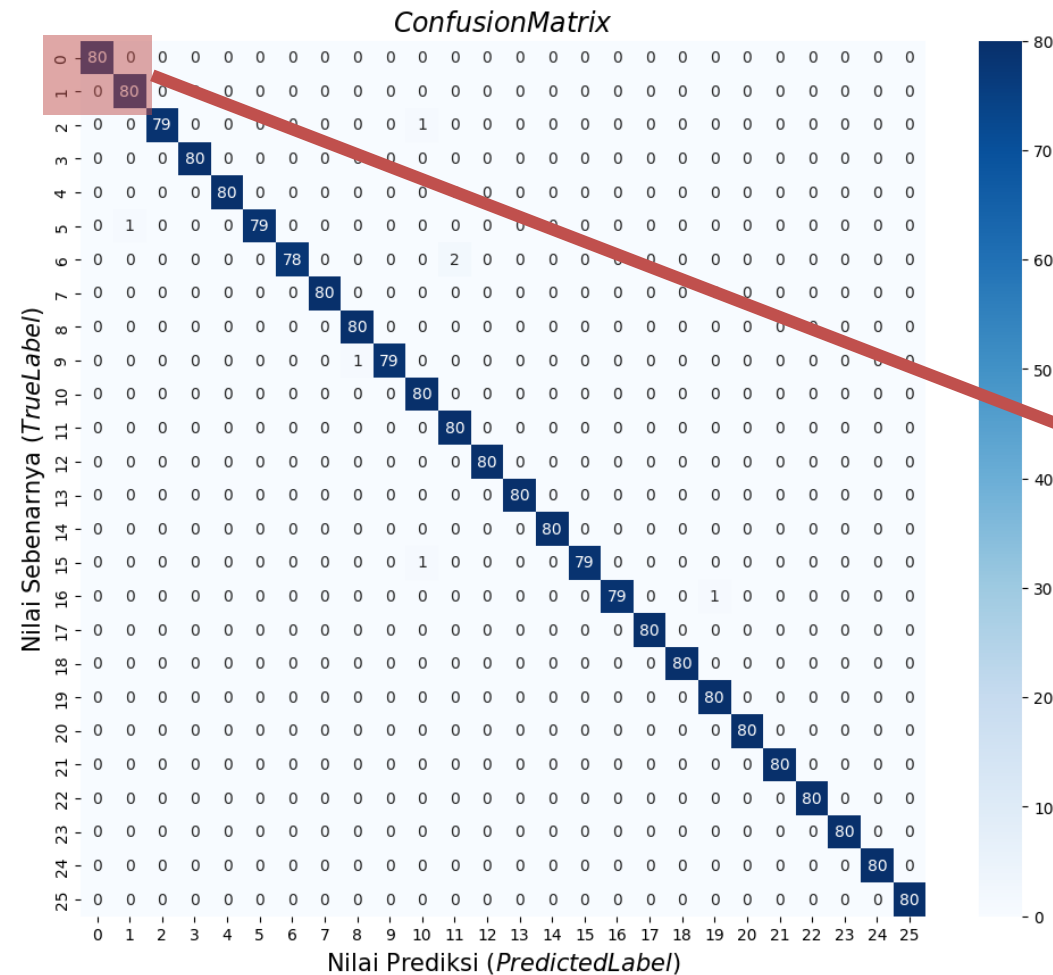


HASIL PEMODELAN JARINGAN SARAF TIRUAN

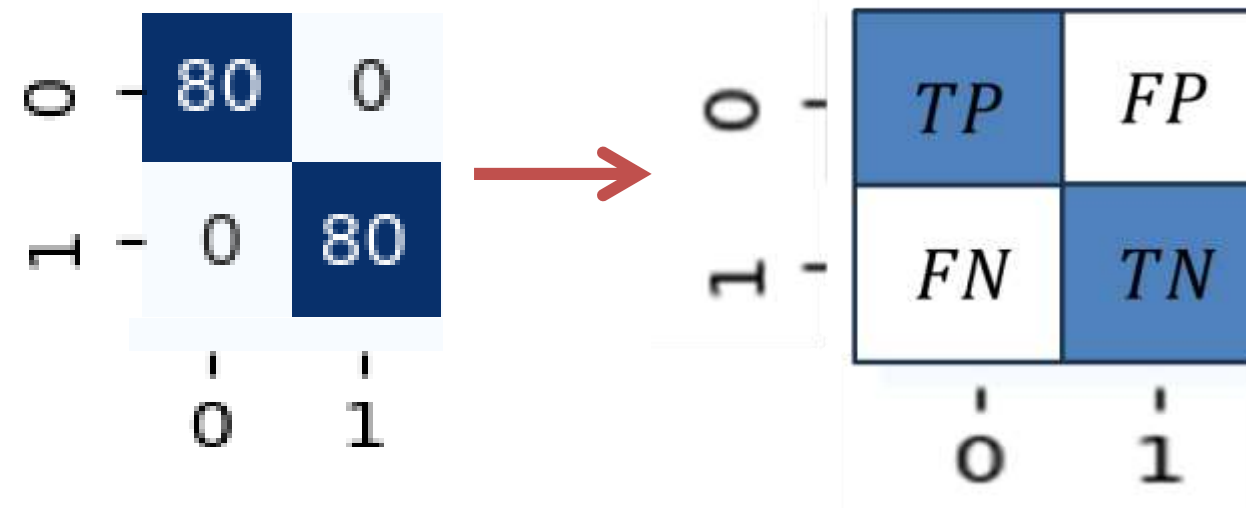
ConfusionMatrix



Cara membaca *Confusion Matrix*



Total *accuracy* pada *Confusin Matrix* 2 klasifikasi (A dan B) :



$$Accuracy = \frac{TP + TN}{(TP + TN) + (FP + FN)} \cdot 100\%$$

$$Accuracy = \frac{80 + 80}{(80 + 80) + (0 + 0)} \cdot 100\% = 100\%$$

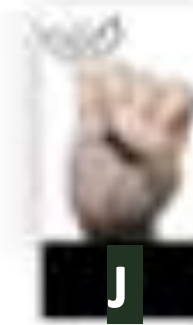
Total *accuracy* pada *Confusin Matrix* 25 klasifikasi:

$$Accuracy = \frac{TP + TN}{(TP + TN) + (FP + FN)} \cdot 100\%$$

$$Accuracy = \frac{80 + 80 + 79 + 80 + 80 + 79 + 78 + 80 + 80 + 79 + 80 + 80 + 80 + 80 + 80 + 79 + 79 + 80 + 80 + 80 + 80 + 80 + 80 + 80 + 80 + 80}{(80 + 80 + 79 + 80 + 80 + 79 + 78 + 80 + 80 + 79 + 80 + 80 + 80 + 80 + 80 + 79 + 79 + 80 + 80 + 80 + 80 + 80 + 80 + 80 + 80 + 80) + (1 + 1 + 1 + 1 + 1 + 2)} \cdot 100\%$$

$$Accuracy = \frac{2073}{2073+7} \cdot 100\% = \frac{2073}{2080} \cdot 100\% = 99,61\% \approx 100\%$$

DEMONSTRASI



<https://pmpk.kemdikbud.go.id/sibi/Abjad>

KESIMPULAN

Telah dikembangkan sebuah sistem komunikasi berupa sarung tangan pintar yang mampu menerjemahkan 26 gestur abjad Sistem Isyarat Bahasa Indonesia (SIBI) dengan *accuracy* 99,83% dan *loss* 0.25%, serta akurasi klasifikasi matriks *confusion* mendekati dengan *accuracy* 100% menggunakan Jaringan Saraf Tiruan (ANN). Meskipun memang dalam penggunaan sarunga tangan secara real sering kali mengalami kesalahan prediksi dikarenakan gerakan (*gesture*) yang mirip pada beberapa Abjad SIBI, contohnya Huruf "M" dan "N".

Analisis fisis dan prinsip kerja sensor berhasil dilakukan melalui kalibrasi sensor GirokopIMU MPU6050 yang mencapai kondisi ideal (percepatan sumbu-Z 1g dan sumbu lain-nya mendekati 0g begitu pun pada sudut rotasi) serta pengukuran sensitivitas pada lima sensor flex yang menunjukkan hubungan yang linear kuat antara sudut tekuk dengan resistansi (rata-rata $R^2 > 0.95$) dan tegangan (rata-rata $R^2 > 0.98$).

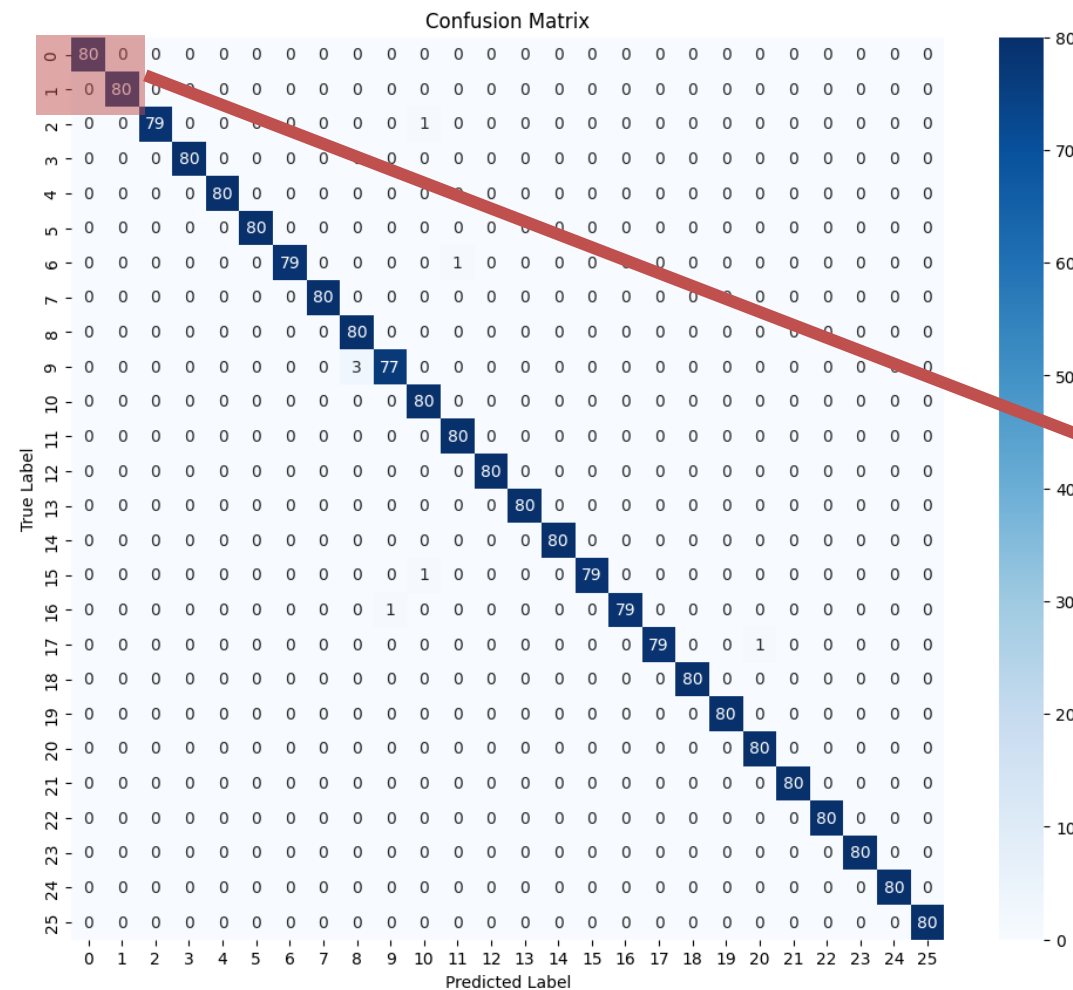
Telah berhasil dibuat sebuah prototipe sarung tangan pintar tunawicara sebagai perangkat asistif (alat bantu) yang cukup fungsional bagi penyandang disabilitas tunawicara.



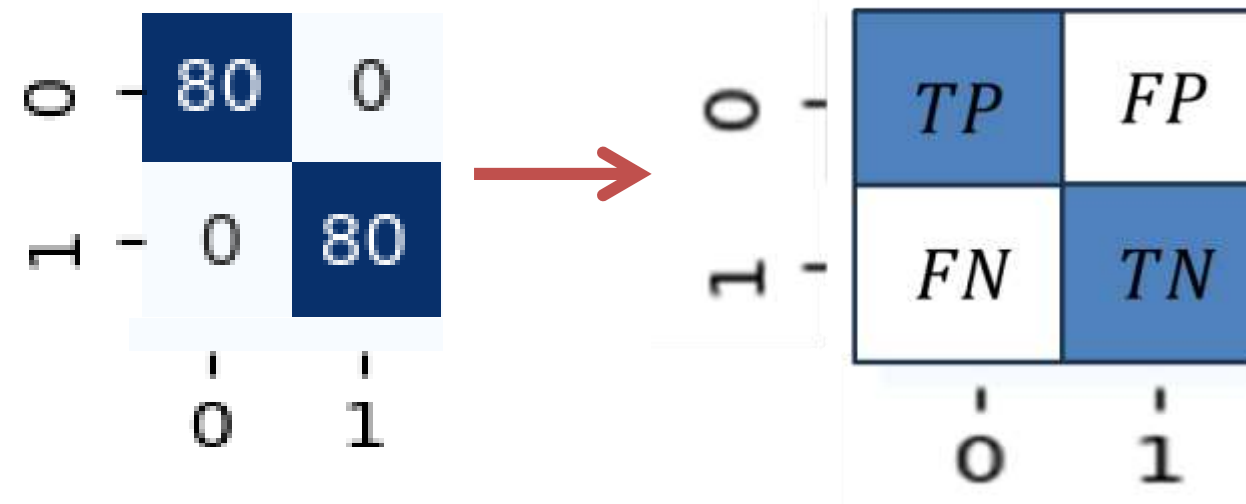
TERIMA KASIH



Cara membaca *Confusion Matrix*



Total accuracy pada *Confusion Matrix* 2x2, 2 klasifikasi (A dan B):



$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \cdot 100\%$$

$$Accuracy = \frac{80 + 80}{80 + 80 + 0 + 0} \cdot 100\% = 100\%$$

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{80}{80 + 0} = 1$$

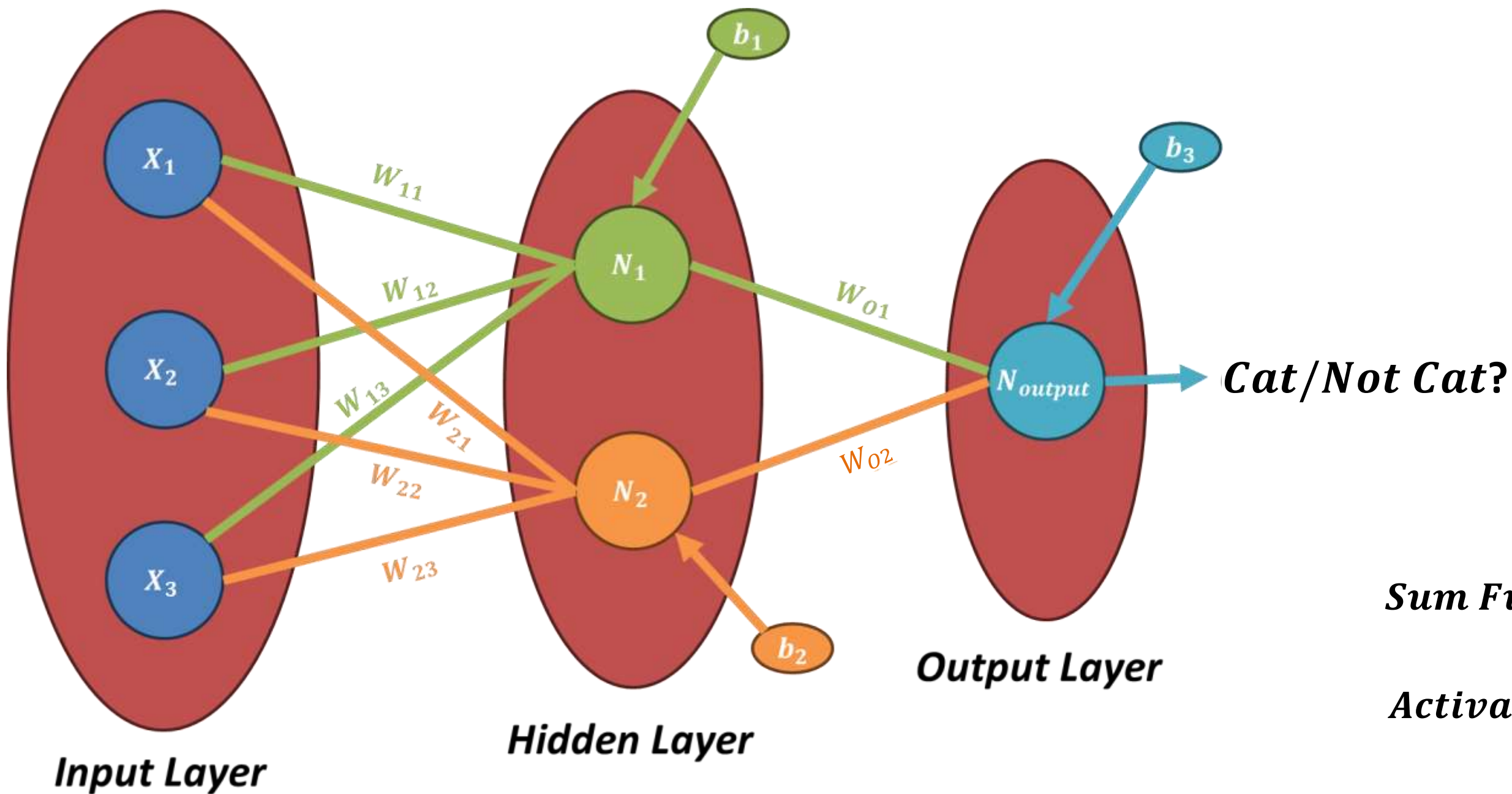
$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{80}{80 + 0} = 1$$

$$F1 - Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

$$F1 - Score = 2 \cdot \frac{1 \cdot 1}{1 + 1} = 1$$

Contoh Sederhana Jaringan Saraf Tiruan sebagai klasifikasi data



	X_1	X_2	X_3	<i>Output</i>
	Ear shape	Face shape	Whiskers	Cat/No cat
	Pointy	Round	Present	1
	Floppy	Nor Round	Present	1
	Floppy	Round	Absent	0
	Pointy	Nor Round	Present	0
	Pointy	Round	Present	1
	Pointy	Round	Absent	1
	Floppy	Nor Round	Absent	0
	Pointy	Round	Absent	1
	Floppy	Round	Absent	0
	Floppy	Round	Absent	0

$$\text{Sum Function} = Z = \sum_{i=1}^N (W_i X_i) + b_i$$

$$\text{Activation Function (Sigmoid)} = f(z) = \frac{1}{1 + e^{-z}}$$

Keterangan: W_i = bobot ke - i
 X_i = data input ke - i
 b_i = bias ke - i

Dalam setiap perhitungan *neuron* pada *Hidden Layer* dan *Output Layer* berlaku perhitungan **Sum Function + Activation Function**

Contoh Sederhana Jaringan Saraf Tiruan sebagai klasifikasi data

	X_1	X_2	X_3	Output
	Ear shape	Face shape	Whiskers	Cat/No cat
1 	Pointy	Round	Present	1
2 	Floppy	Nor Round	Present	1
3 	Floppy	Round	Absent	0
4 	Pointy	Nor Round	Present	0
5 	Pointy	Round	Present	1
6 	Pointy	Round	Absent	1
7 	Floppy	Nor Round	Absent	0
8 	Pointy	Round	Absent	1
9 	Floppy	Round	Absent	0
10 	Floppy	Round	Absent	0

Pointy = 1; *Floppy* = 0

Round = 1; *Not Round* = 0

Present = 1; *Absent* = 0

Dengan menggunakan data ke-1:

$X_1 = 1$; $X_2 = 1$; $X_3 = 1$

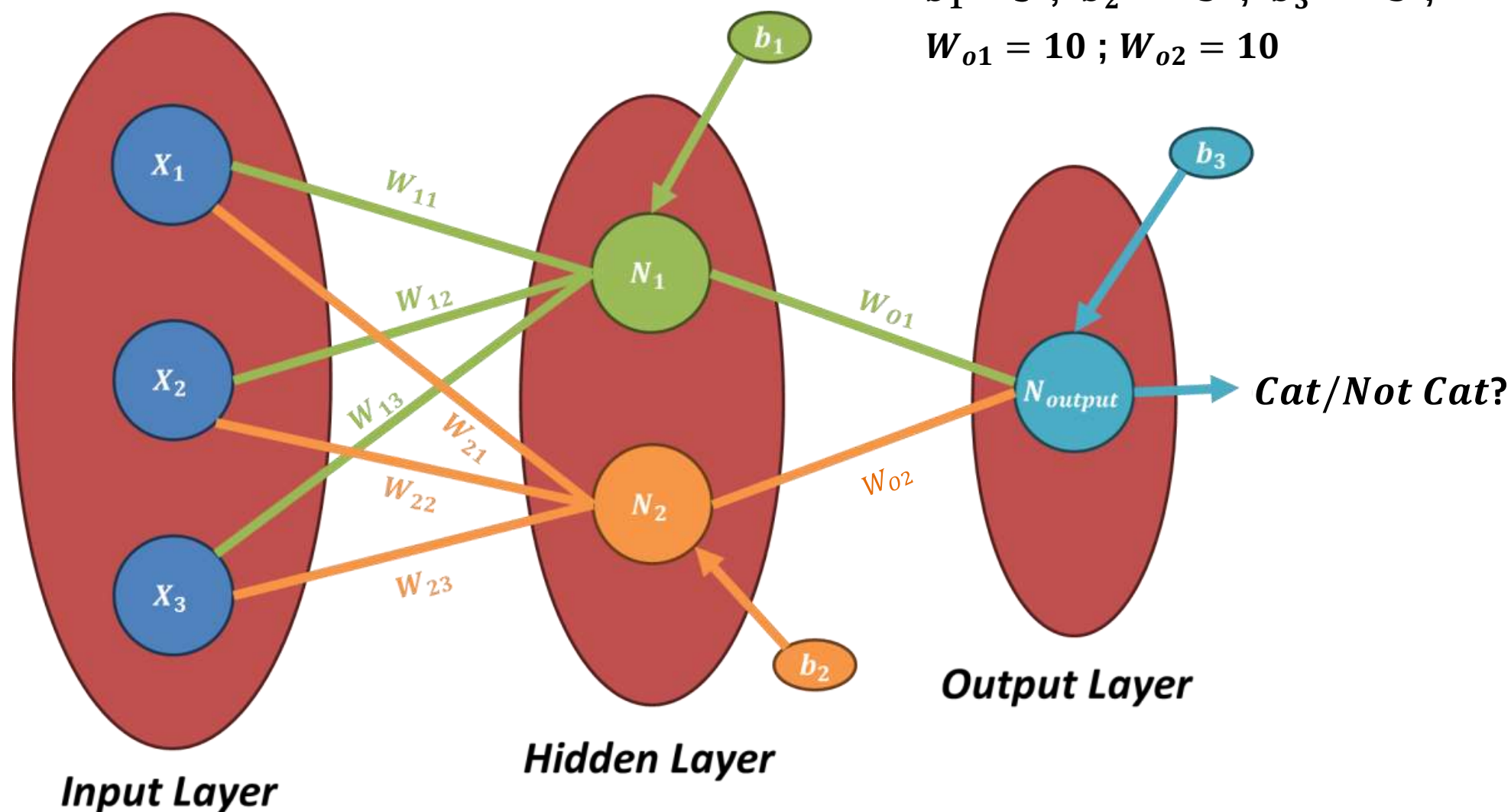
Parameter yang digunakan:

$W_{11} = 10$; $W_{12} = 10$; $W_{13} = 0$;

$W_{21} = -10$; $W_{22} = 0$; $W_{23} = 10$;

$b_1 = 5$; $b_2 = -5$; $b_3 = -5$;

$W_{01} = 10$; $W_{02} = 10$



$$\text{Sum Function} = Z = \sum_{i=1}^N (W_i X_i) + b_i$$

$$\text{Activation Function (Sigmoid)} = f(z) = \frac{1}{1 + e^{-z}}$$

Perhitungan neuron ke-1 pada *Hidden Layer* (N_1)

$$Z_{h1} = (W_{11}X_1 + W_{12}X_2 + W_{13}X_3) + b_1$$

$$Z_{h1} = (10 \cdot 1 + 10 \cdot 1 + 0 \cdot 1) + (-15) = 5$$

$$N_1 = f(Z_{h1}) = \frac{1}{1 + e^{-Z_{h1}}} = \frac{1}{1 + e^{-(5)}} = 0,9933$$

Perhitungan neuron ke-2 pada *Hidden Layer* (N_2)

$$Z_{h2} = (W_{21}X_1 + W_{22}X_2 + W_{23}X_3) + b_2$$

$$Z_{h2} = (-10 \cdot 1 + 0 \cdot 1 + 10 \cdot 1) + (-5) = -5$$

$$N_2 = f(Z_{h2}) = \frac{1}{1 + e^{-Z_{h2}}} = \frac{1}{1 + e^{-(-5)}} = 0,0067$$

Perhitungan neuron output pada *Output Layer*

$$Z_{N_{\text{output}}} = (W_{01}N_1 + W_{02}N_2) + b_3$$

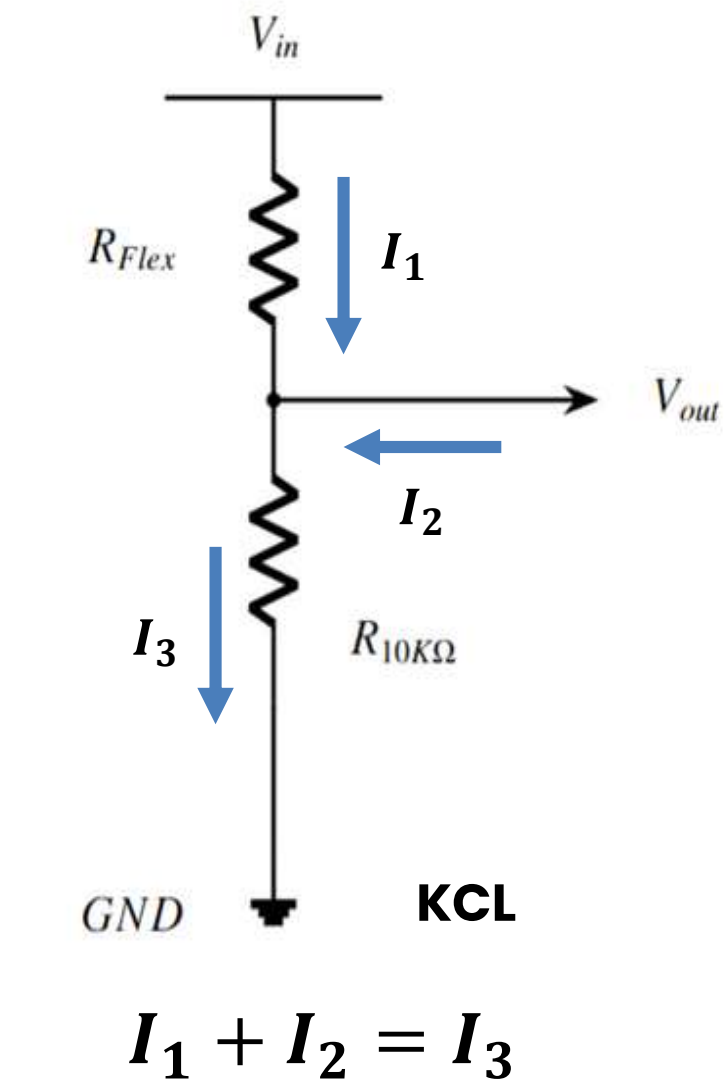
$$Z_{N_{\text{output}}} = (10 \cdot 0,9933 + 10 \cdot 0,0067) + (-5) = 5$$

$$\text{Output} = f(Z_{N_{\text{output}}}) = \frac{1}{1 + e^{-Z_{N_{\text{output}}}}} = \frac{1}{1 + e^{-(5)}} = 0,9933 \approx 1 = \text{cat}$$



Pembahasan Fisis Sensor Flex (Resistor Fleksible)

Rangkaian Pemasangan Sensor *Flex*



$$\frac{V_{in} - V_{out}}{R_{flex}} = \frac{V_{out}}{R_{10K\Omega}}$$

Pembagi Tegangan

$$V_{out} = V_{in} \cdot \frac{R_{10K\Omega}}{R_{flex} + R_{10K\Omega}}$$

$$\frac{R_{flex} + R_{10K\Omega}}{R_{10K\Omega}} = \frac{V_{in}}{V_{out}}$$

$$R_{flex} + R_{10K\Omega} = R_{10K\Omega} \cdot \frac{V_{in}}{V_{out}}$$

$$R_{flex} = R_{10K\Omega} \cdot \frac{V_{in}}{V_{out}} - R_{10K\Omega}$$

$$R_{flex} = R_{10K\Omega} \cdot \left(\frac{V_{in}}{V_{out}} - 1 \right)$$

Hukum OHM

$$I \cdot R = V$$

$$\frac{V_{out}}{R_{10K\Omega}} = V_{in} \cdot \frac{1}{R_{flex} + R_{10K\Omega}}$$

$$I_{out} = \frac{V_{in}}{R_{flex} + R_{10K\Omega}}$$

$$I_{out} \cdot (R_{flex} + R_{10K\Omega}) = V_{in}$$

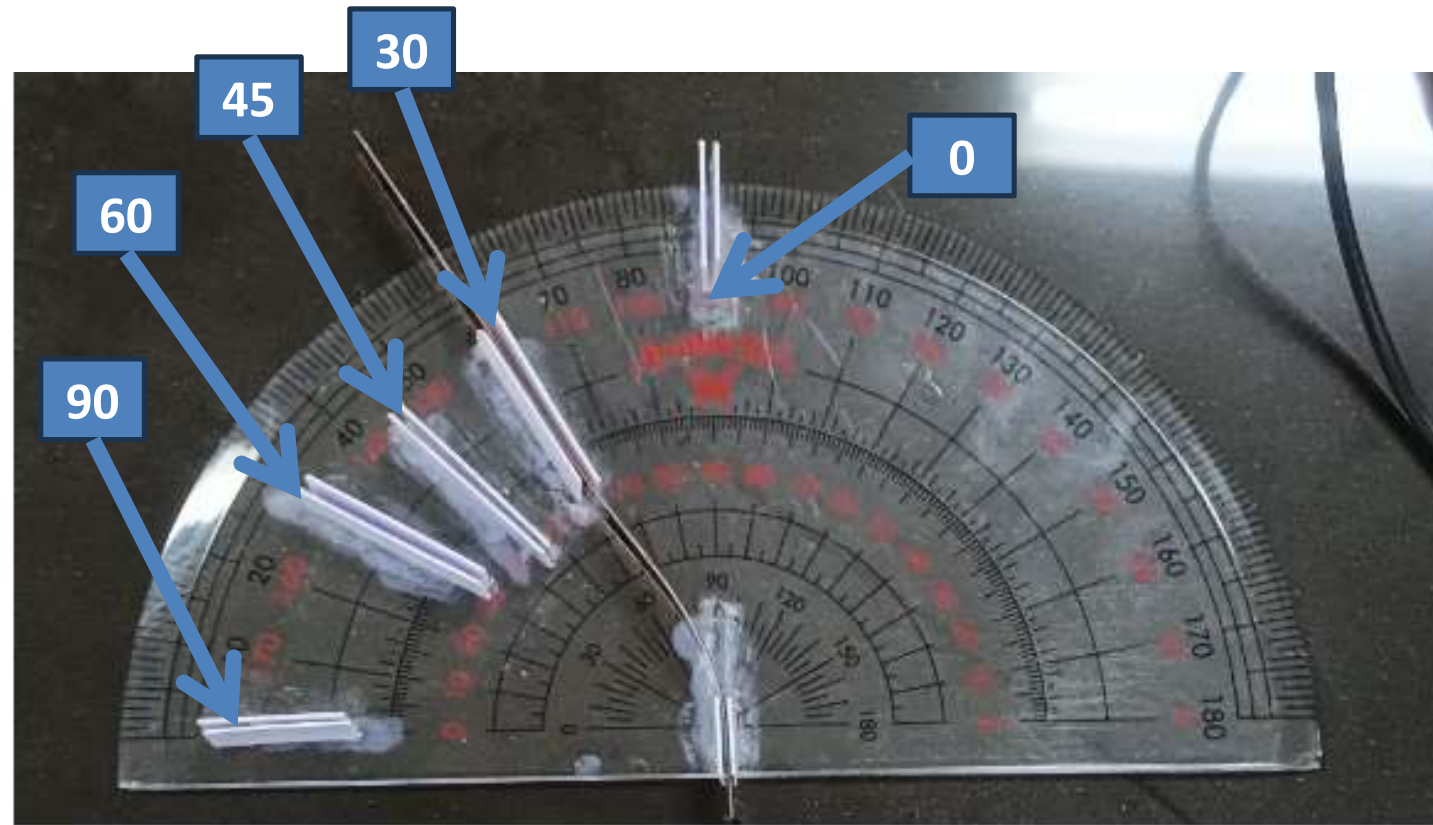
Code For Sensor Flex

```
const float TeganganInput = 4.44;           // Tegangan input dalam volt (4.44V untuk ESP32)
const float R1 = 10.0;                      // Nilai resistor tetap dalam kOhm
```

```
// Hitung tegangan untuk setiap pembacaan (ADC 12-bit, nilai 0-4095)
float tegangan = nilaiSensor * (TeganganInput / 4095.0); // Konversi ke tegangan (0-4.44V)
totalTegangan += tegangan;

// Hitung resistansi untuk setiap pembacaan
if (tegangan > 0) { // Hindari pembagian dengan nol
    float resistansi = (TeganganInput * R1 / tegangan) - R1; // Hitung resistansi dalam kOhm
    totalResistansi += resistansi;
}
```

Pembagi Tegangan $\longrightarrow$
$$V_{out} = V_{in} \cdot \frac{R_{10K\Omega}}{R_{flex} + R_{10K\Omega}}$$
 $\longrightarrow$
$$R_{flex} = R_{10K\Omega} \cdot \frac{V_{in}}{V_{out}} - R_{10K\Omega}$$



Gambar 3.8. cara pengukuran sudut sensor *flex* pada busur derajat



Gambar 3.9. Pengukuran resistansi sensor *flex* pada multimeter



Gambar 3.10. Pengukuran resistansi sensor *flex* pada serial monitor pemrograman Arduino

Sudut (θ°)	Sensor Flex ke-1 ($K\Omega$)		Sensor Flex ke-2 ($K\Omega$)	
	Multimeter	Serial Monitor	Multimeter	Serial Monitor
0	11,97	13,48	11,17	12,69
30	14,73	17,01	12,97	14,43
45	17,23	19,43	14,77	16,74
60	19,53	21,70	16,47	18,07
90	23,30	27,17	19,77	22,11

Proses Kalibrasi Sensor Giroskop IMU MPU6050

Tabel 5.1. Data Kalibrasi Sensor IMU untuk Berbagai Konfigurasi.



Konfigurasi	Jenis Data	Parameter Sumbu Akselerometer			Parameter Sumbu Giroskop		
		X (ax)	Y (ay)	Z (az)	X (gx)	Y (gy)	Z (gz)
		[m/s ²]	[m/s ²]	[m/s ²]	[rad/s]	[rad/s]	[rad/s]
1	Parameter Offset Kalibrasi	-0.07	0.01	-0.03	0.02	0.02	0
	Data Output Gyro	0.14	-0.03	1.07	-0.04	-0.04	0
2	Parameter Offset Kalibrasi	-0.14	0.03	-0.07	0	0	0
	Data Output Gyro	0.19	-0.03	1.11	-0.01	-0.02	0
3	Parameter Offset Kalibrasi	0.14	-0.02	0.02	0	0	0
	Data Output Gyro	-0.13	0.01	1.03	-0.02	-0.02	0
4	Parameter Offset Kalibrasi	0.07	-0.03	0.04	0	0	0
	Data Output Gyro	-0.07	0.02	1.00	-0.02	-0.02	0
5	Parameter Offset Kalibrasi	0.03	-0.04	0.04	-0.01	-0.01	0
	Data Output Gyro	0.14	-0.03	1.07	-0.04	-0.04	0
6	Parameter Offset Kalibrasi	0.02	-0.02	0.04	-0.01	-0.01	0
	Data Output Gyro	-0.03	0.01	1.00	0	-0.01	0
7	Parameter Offset Kalibrasi	0	-0.02	0.04	-0.01	-0.01	0
	Data Output Gyro	0.01	0	1.00	0	-0.01	0

```
// offset dari kalibrasi
float ax_offset = 0.00; // g
float ay_offset = -0.02; // g
float az_offset = 0.04; // g
float gx_offset = -0.01; // rad/s
float gy_offset = -0.01; // rad/s
float gz_offset = 0.00; // rad/s
```


Proses Deteksi Data Sensor Girooskop IMU MPU6050

```
// Konfigurasi sensor
mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
```

Scale Range LSB Sensitivity (RAW)

$$\pm 2g = 16384 \text{ LSB/g}$$

$$a_{(x,y,z)} [g] = \frac{RAW_{(x,y,z)}}{Sensitivity}$$

$$a_{(x,y,z)} [g] = \frac{16908}{16384}$$

$$a_{(x,y,z)} [g] = 1,03g \approx 1g$$

$$1g = 9,81 \text{ m/s}^2$$

Tabel. Contoh data RAW pada percepatan sumbu (X,Y,Z)

rawX	rawY	rawZ
332	820	16808
384	922	16872
398	954	16858
410	916	16882
370	920	16910
392	908	16858
398	926	16908
344	884	16910
392	922	16874
388	926	16846
386	902	16888
390	930	16886
428	954	16830
340	904	16828
356	912	16876
388	962	16932
372	928	16926
358	944	16888
368	932	16880

4.18 Registers 59 to 64 – Accelerometer Measurements

ACCEL_XOUT_H, ACCEL_XOUT_L, ACCEL_YOUT_H, ACCEL_YOUT_L, ACCEL_ZOUT_H, and ACCEL_ZOUT_L

Type: Read Only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT[7:0]							

Description:

These registers store the most recent accelerometer measurements.

Accelerometer measurements are written to these registers at the Sample Rate as defined in Register 25.

The accelerometer measurement registers, along with the temperature measurement registers, gyroscope measurement registers, and external sensor data registers, are composed of two sets of registers: an internal register set and a user-facing read register set.

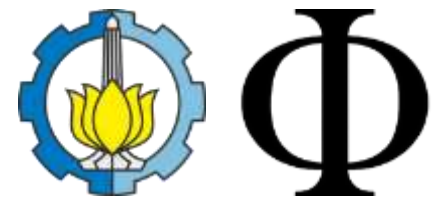
The data within the accelerometer sensors' internal register set is always updated at the Sample Rate. Meanwhile, the user-facing read register set duplicates the internal register set's data values whenever the serial interface is idle. This guarantees that a burst read of sensor registers will read measurements from the same sampling instant. Note that if burst reads are not used, the user is responsible for ensuring a set of single byte reads correspond to a single sampling instant by checking the Data Ready interrupt.

Each 16-bit accelerometer measurement has a full scale defined in *ACCEL_FS* (Register 28). For each full scale setting, the accelerometers' sensitivity per LSB in *ACCEL_xOUT* is shown in the table below.

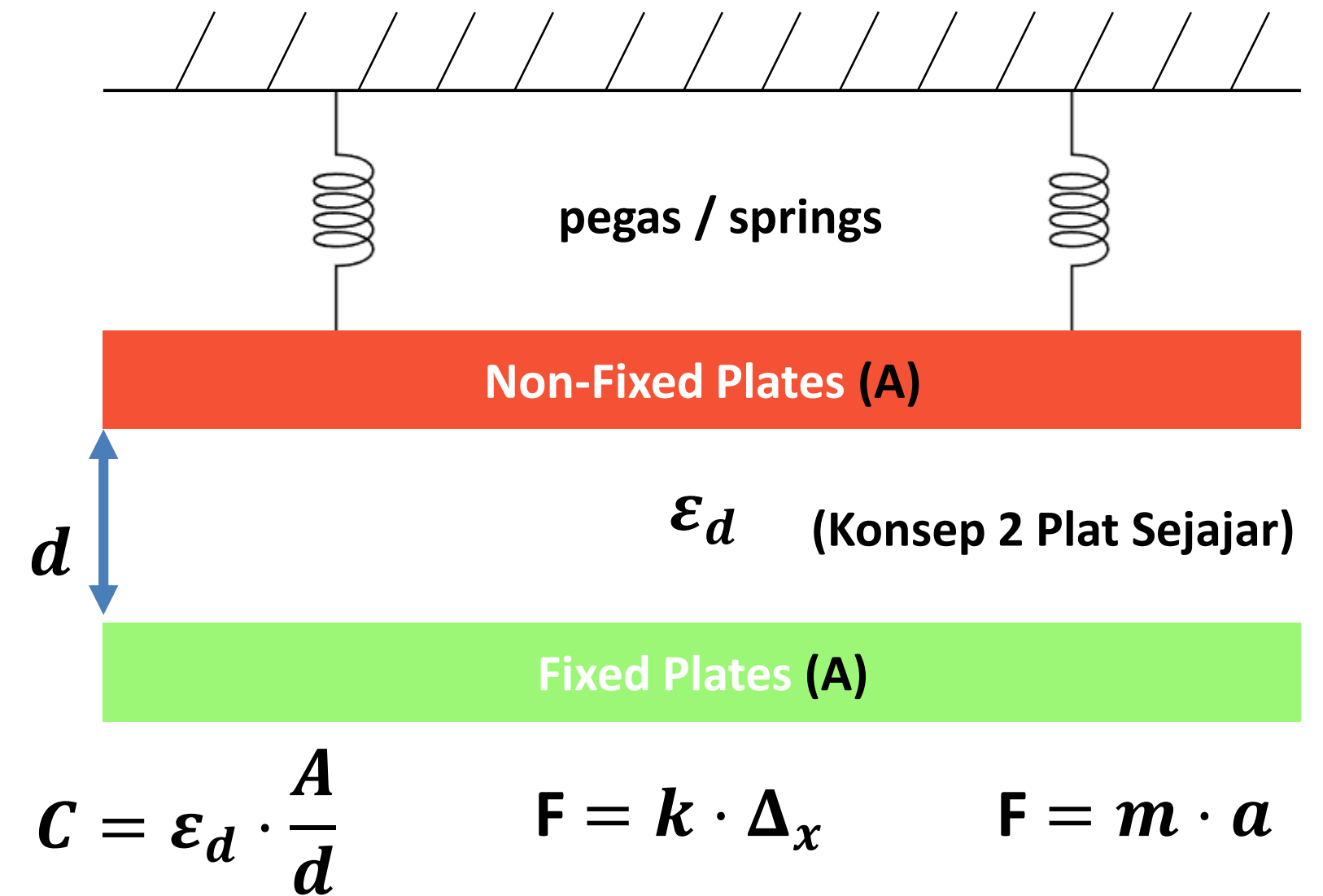
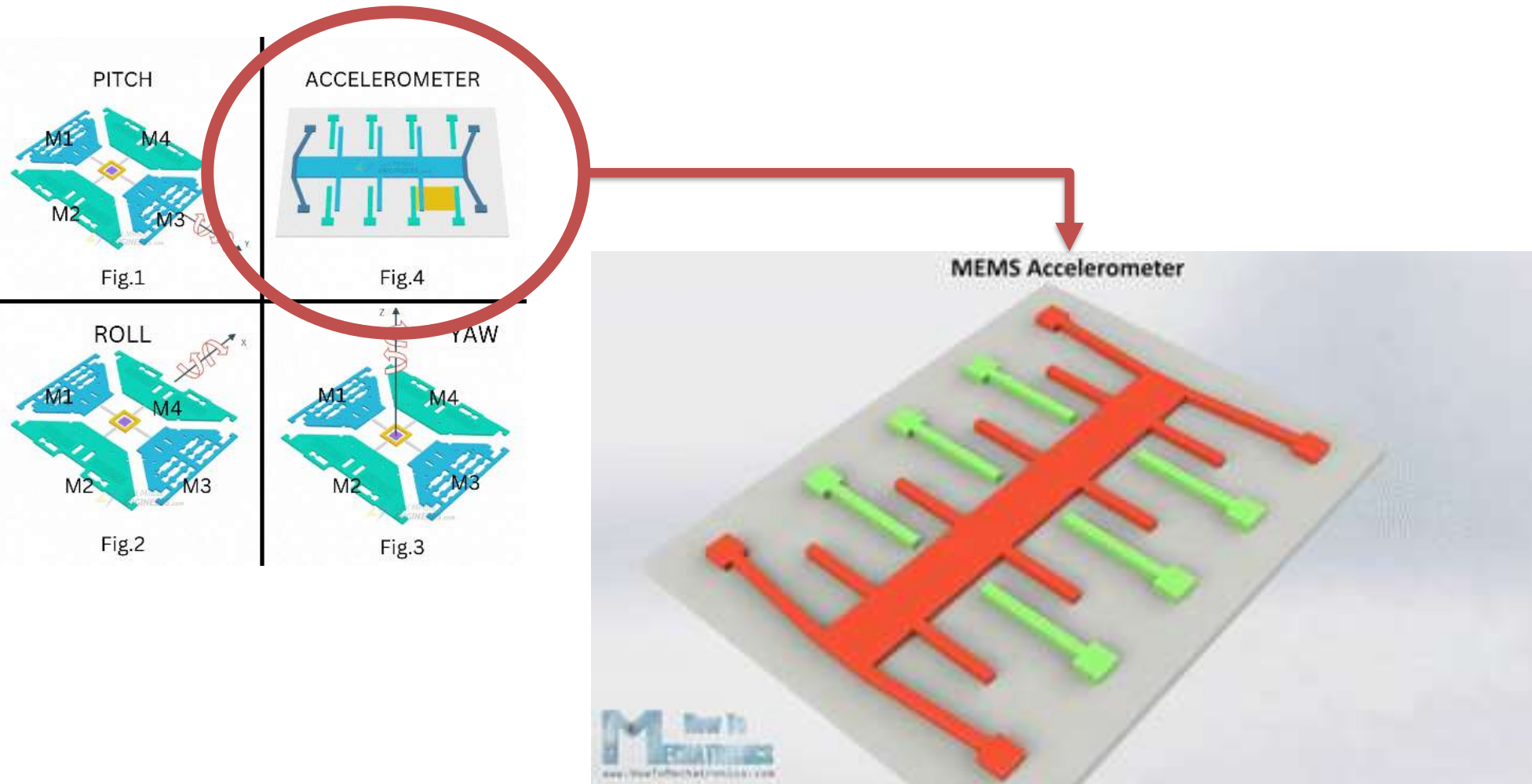
AFS_SEL	Full Scale Range	LSB Sensitivity
0	$\pm 2g$	16384 LSB/g
1	$\pm 4g$	8192 LSB/g
2	$\pm 8g$	4096 LSB/g
3	$\pm 16g$	2048 LSB/g

Parameters:

<i>ACCEL_XOUT</i>	16-bit 2's complement value. Stores the most recent X axis accelerometer measurement.
<i>ACCEL_YOUT</i>	16-bit 2's complement value. Stores the most recent Y axis accelerometer measurement.
<i>ACCEL_ZOUT</i>	16-bit 2's complement value. Stores the most recent Z axis accelerometer measurement.



Cara Kerja Fisis dalam MPU6050

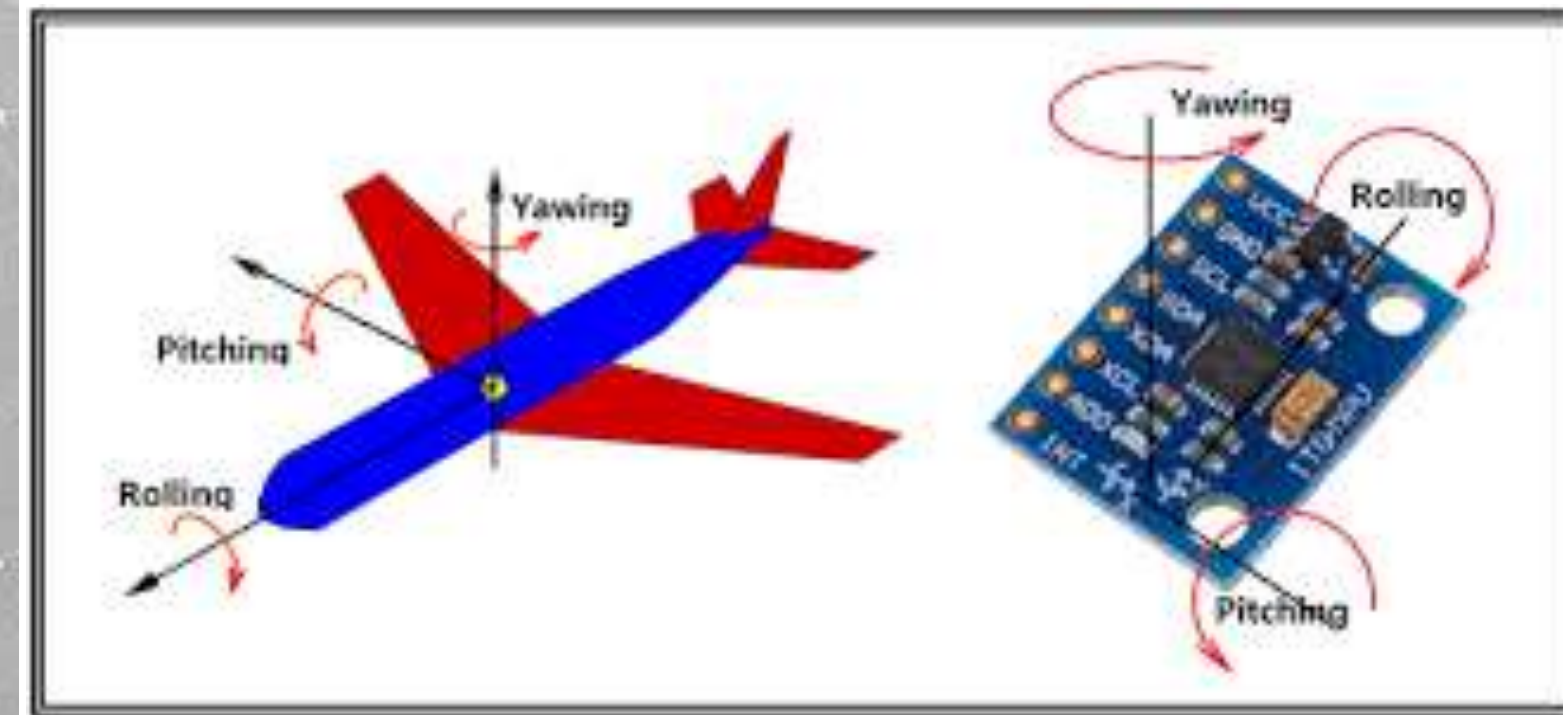
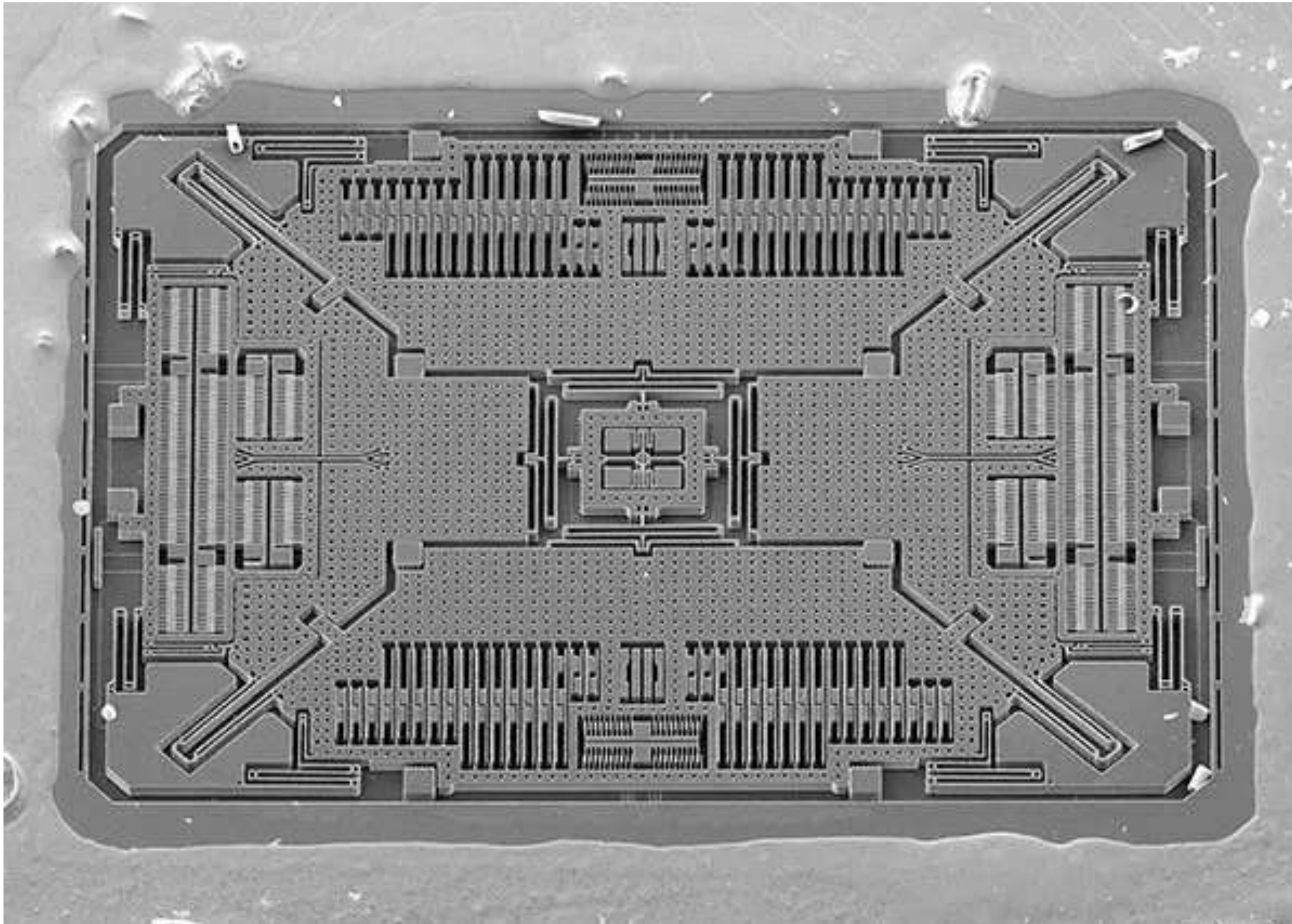


Sehingga,

$$\begin{array}{l}
 d = \epsilon_d \cdot \frac{A}{C} \\
 F = k \cdot \Delta_d
 \end{array}
 \longrightarrow
 \begin{array}{l}
 F = k \cdot \Delta_d \\
 F = k \cdot (d_2 - d_1) \\
 F = k \cdot (\epsilon_d \cdot \frac{A}{C_2} - \epsilon_d \cdot \frac{A}{C_1})
 \end{array}
 \longrightarrow
 \begin{array}{l}
 F = k \cdot (\epsilon_d \cdot \frac{A}{C_2} - \epsilon_d \cdot \frac{A}{C_1}) \\
 F = k \cdot \epsilon_d \cdot A(\frac{1}{C_2} - \frac{1}{C_1})
 \end{array}
 \longrightarrow
 \begin{array}{l}
 m \cdot a = k \cdot \epsilon_d \cdot A(\frac{1}{C_2} - \frac{1}{C_1}) \\
 a_{(x,y,z)} = \frac{k \cdot \epsilon_d \cdot A(\frac{1}{C_2} - \frac{1}{C_1})}{m}
 \end{array}$$



Komponen mikro MPU6050

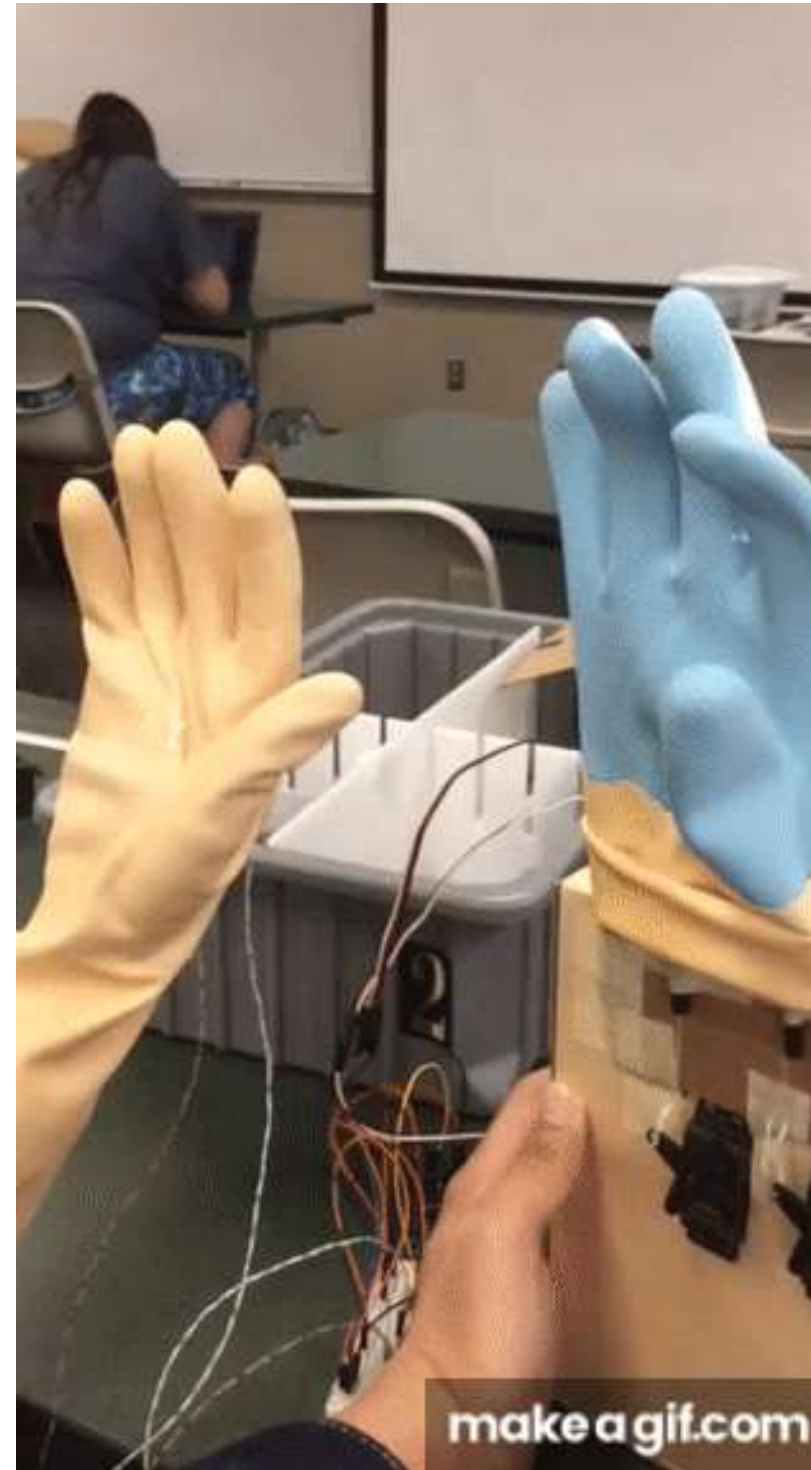




Contoh Training ANN

```
Epoch 1/25
260/260 ————— 9s 6ms/step - accuracy: 0.0668 - loss: 3.5948 - val_accuracy: 0.3423 - val_loss: 2.8790
Epoch 2/25
260/260 ————— 1s 3ms/step - accuracy: 0.1726 - loss: 3.0026 - val_accuracy: 0.6808 - val_loss: 2.3102
Epoch 3/25
260/260 ————— 1s 3ms/step - accuracy: 0.3155 - loss: 2.6185 - val_accuracy: 0.7870 - val_loss: 1.9320
Epoch 4/25
260/260 ————— 1s 3ms/step - accuracy: 0.4304 - loss: 2.2743 - val_accuracy: 0.8524 - val_loss: 1.6420
Epoch 5/25
260/260 ————— 3s 10ms/step - accuracy: 0.5162 - loss: 2.0286 - val_accuracy: 0.8635 - val_loss: 1.3788
Epoch 6/25
260/260 ————— 4s 8ms/step - accuracy: 0.5728 - loss: 1.8139 - val_accuracy: 0.8856 - val_loss: 1.1659
Epoch 7/25
260/260 ————— 2s 7ms/step - accuracy: 0.6409 - loss: 1.6052 - val_accuracy: 0.9231 - val_loss: 0.9984
Epoch 8/25
260/260 ————— 1s 4ms/step - accuracy: 0.6638 - loss: 1.4518 - val_accuracy: 0.9558 - val_loss: 0.8537
Epoch 9/25
260/260 ————— 1s 4ms/step - accuracy: 0.7120 - loss: 1.2999 - val_accuracy: 0.9736 - val_loss: 0.7485
Epoch 10/25
260/260 ————— 1s 4ms/step - accuracy: 0.7256 - loss: 1.2060 - val_accuracy: 0.9798 - val_loss: 0.6572
Epoch 11/25
260/260 ————— 1s 4ms/step - accuracy: 0.7474 - loss: 1.1316 - val_accuracy: 0.9865 - val_loss: 0.5888
Epoch 12/25
260/260 ————— 1s 5ms/step - accuracy: 0.7892 - loss: 1.0057 - val_accuracy: 0.9885 - val_loss: 0.5239
Epoch 13/25
260/260 ————— 1s 4ms/step - accuracy: 0.7857 - loss: 0.9557 - val_accuracy: 0.9904 - val_loss: 0.4751
Epoch 14/25
260/260 ————— 1s 4ms/step - accuracy: 0.8199 - loss: 0.8810 - val_accuracy: 0.9913 - val_loss: 0.4311
Epoch 15/25
260/260 ————— 2s 7ms/step - accuracy: 0.8230 - loss: 0.8309 - val_accuracy: 0.9913 - val_loss: 0.3962
```

Contoh Aplikatif lainnya



Hyperparameter



Learning rate: A parameter that controls the size of the learning step taken during model optimization. It's important to choose carefully to avoid convergence that's either too fast or too slow.



Number of epochs: The number of iterations through the entire training dataset. It should be chosen so that the model learns well without overfitting or underfitting.



Batch size: The number of samples given to the model at each training iteration. It affects the convergence and efficiency of the training process.



Number of layers and neurons: The neural network structure, including the number of layers and neurons in each layer. It affects the model's ability to learn complex patterns.



Activation function: The choice of activation function in each network layer. It influences the network's ability to learn good representations from the data.



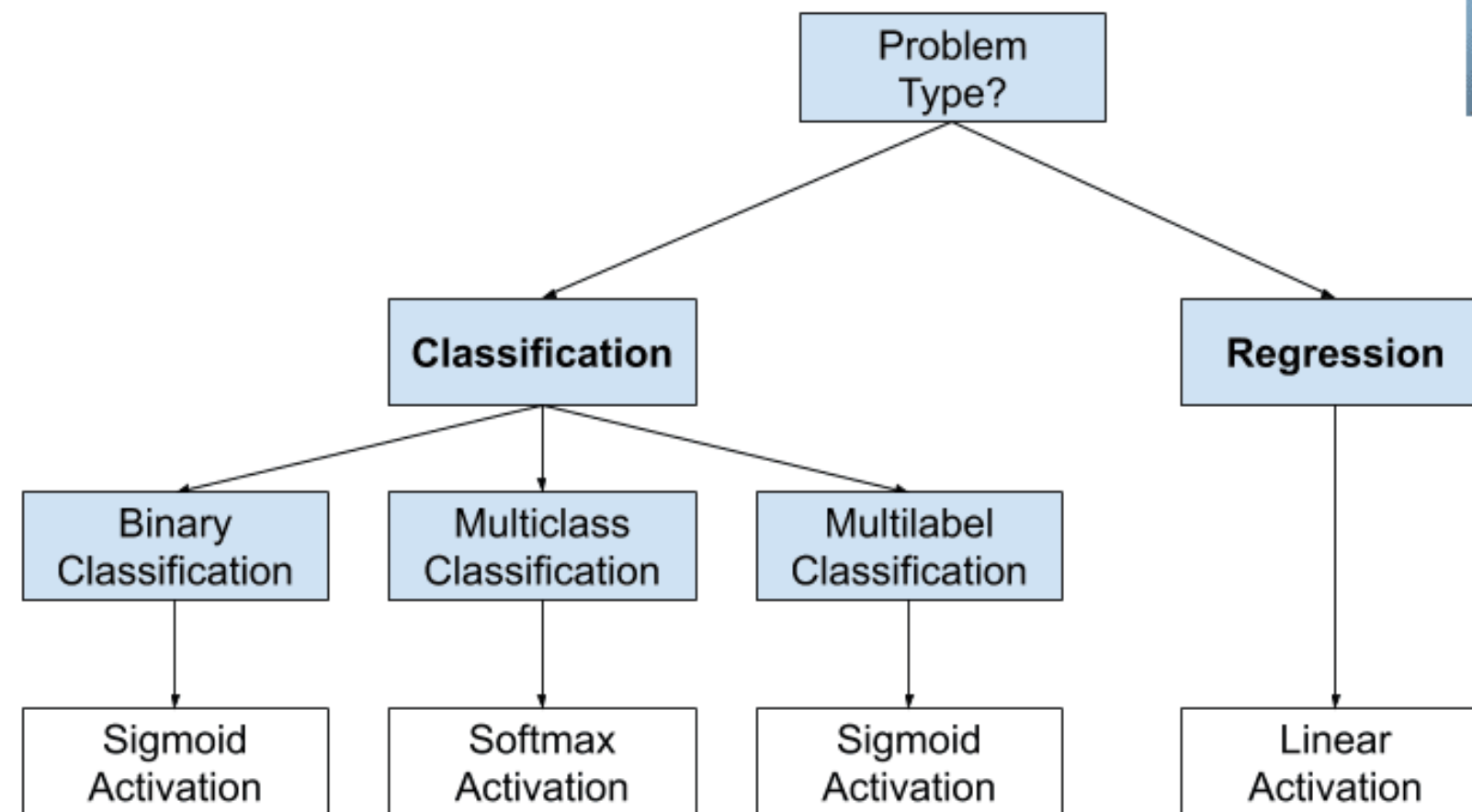
Regularization: Parameters that control regularization techniques such as dropout or L1/L2 penalties. They prevent overfitting by reducing the complexity of the model.



Optimizer: The algorithm used to optimize the loss function during training, such as SGD, Adam, RMSProp, etc. It affects the speed and quality of model convergence.

Macam-macam Activation Function

How to Choose an Output Layer Activation Function



MachineLearningMastery.com

Linear Activation Function

$$f(x) = x$$

Tanh Function

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

ReLU

$$f(x) = \max(0, x)$$

Leaky ReLU and Parametric ReLU

$$f(x) = x \text{ if } x > 0, \text{ else } f(x) = \alpha x$$

Softmax Function

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Macam-macam Activation Function

Linear Activation Function

$$f(x) = x$$

Tanh Function

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

ReLU

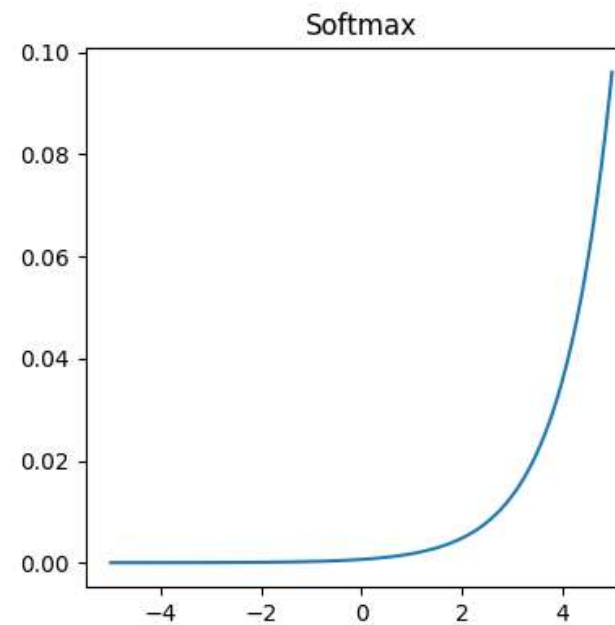
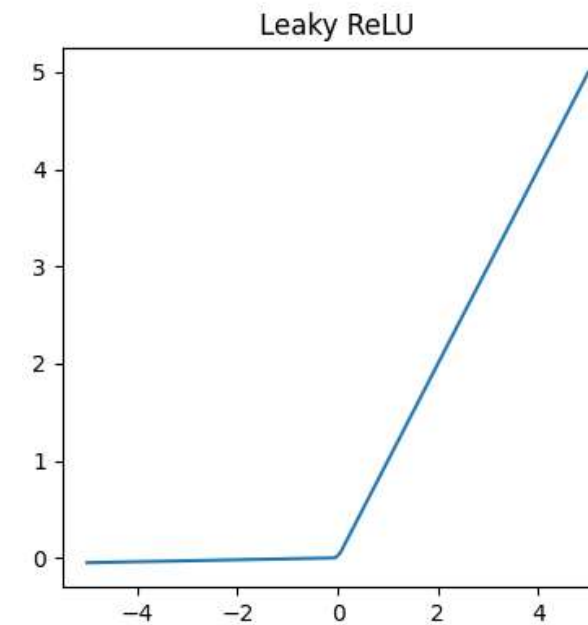
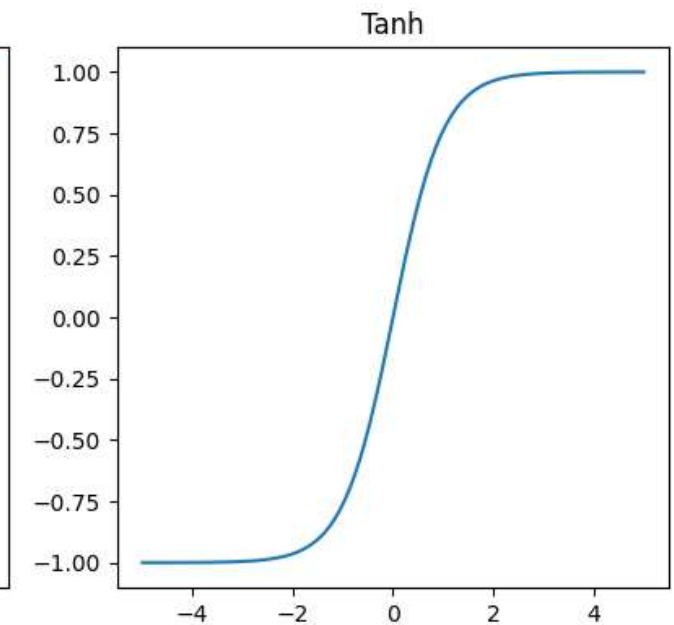
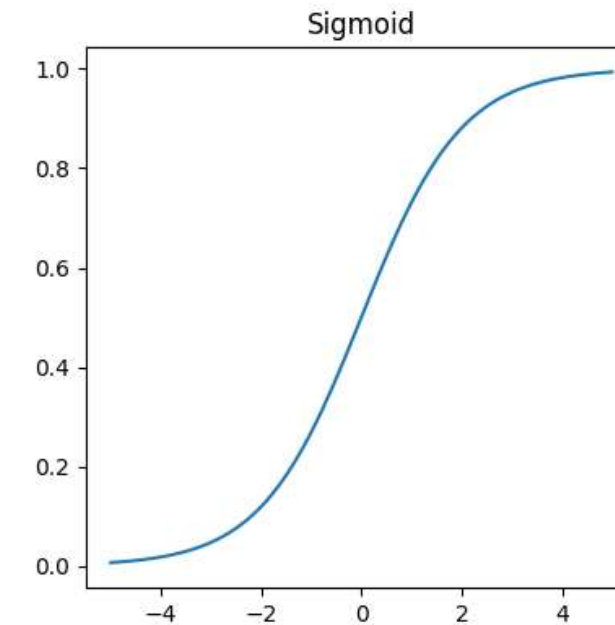
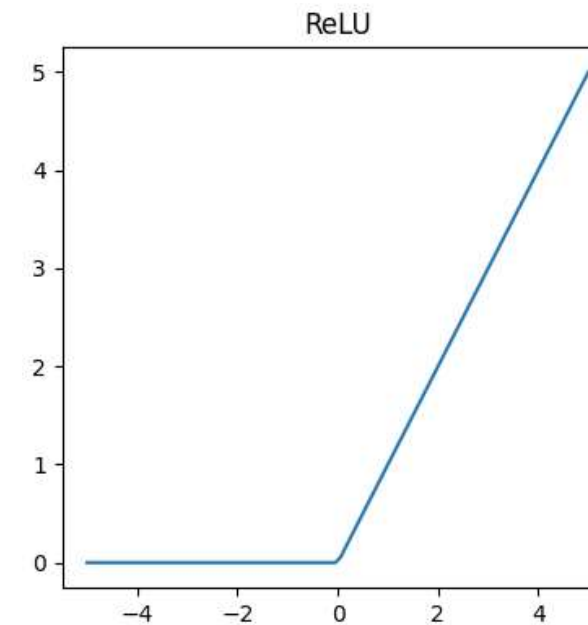
$$f(x) = \max(0, x)$$

Leaky ReLU and Parametric ReLU

$$f(x) = x \text{ if } x > 0, \text{ else } f(x) = \alpha x$$

Softmax Function

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



Optimizer Function with Adam(Adaptive Moment Estimation)

- First Momentum: $m_t = \beta_1 \times m_{t-1} + (1 - \beta_1) \times g_t$
- Second Moment : $v_t = \beta_2 \times v_{t-1} + (1 - \beta_2) \times (g_t)^2$
- Bias Correction :
- $\widehat{m}_t = \beta_1 \cdot m_{t+1} + (1 - \beta_1) \cdot g_t$
- $\widehat{v}_t = \beta_2 \cdot v_{t+1} + (1 - \beta_2) \cdot g_t^2$
- Update Rule: $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v}_t + \epsilon}} \cdot \widehat{m}_t$
- Where:
 - m_t and v_t are exponentially decaying moving averages of the first moment (mean) and second moment (uncentered variance) of the gradients, respectively.
 - β_1 and β_2 are exponential decay rates for the moving averages (typically close to 1).
 - ϵ is a small constant to prevent division by zero.
 - $\{\widehat{m}\}_{t+1}$ and $\{\widehat{v}\}_{t+1}$ are bias-corrected estimates of the first and second moments, respectively.

Alur Pelatihan ANN

- Step 0= Initialize the weights and bias
- Step 1= Feed the training sample through the network and get the final output
- Step 2= Compute the error for output unit

$$\delta_k = (t_k - y_k)f'(y_{in_k})$$

- Step 3= Calculate the weight correction for each output unit

$$\Delta W_{jk} = \eta \delta_k z_j$$

- Step 4= Propagate the delta errors using weights of hidden units where the delta input for the j^{th} hidden unit is:

$$\delta_{in_j} = \sum_{k=1}^m \delta_k w_{jk} f'(z_{in_j})$$

Alur Pelatihan ANN

- Where:

$$f'(z_{in_j}) = f(z_{in_j}) [1 - f(z_{in_j})]$$

- Step 5= Calculate the weight correction term for the hidden units

$$\Delta W_{ij} = \eta \delta_j X_i$$

- Step 6= Update the weights:

$$W_{jk}(\text{new}) = W_{jk}(\text{old}) + \Delta W_{jk}$$

- Step 7= Test for stepping (maximum cycles, small changes, etc)

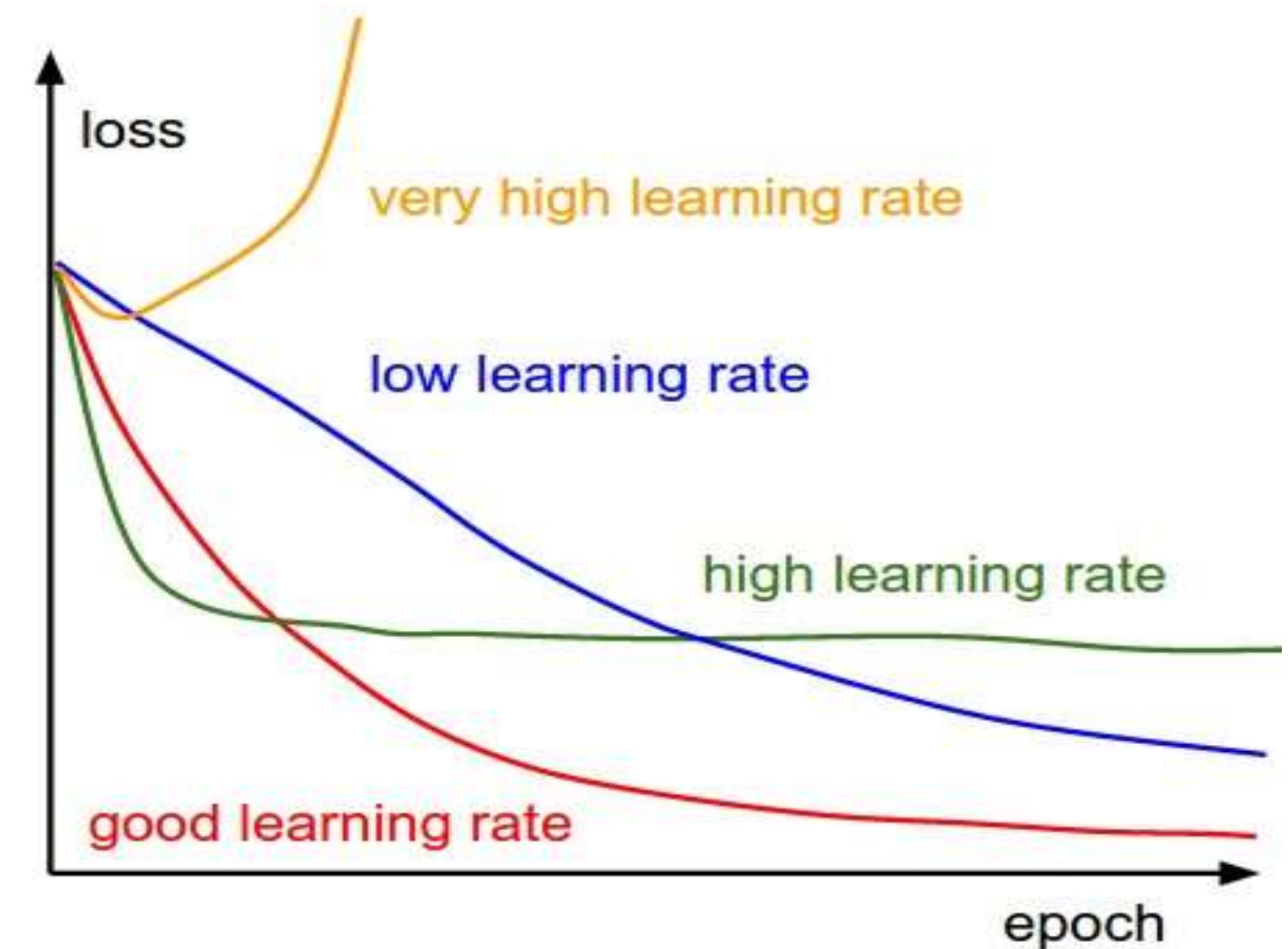
Loss Function

Sparse Categorical Cross-Entropy

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

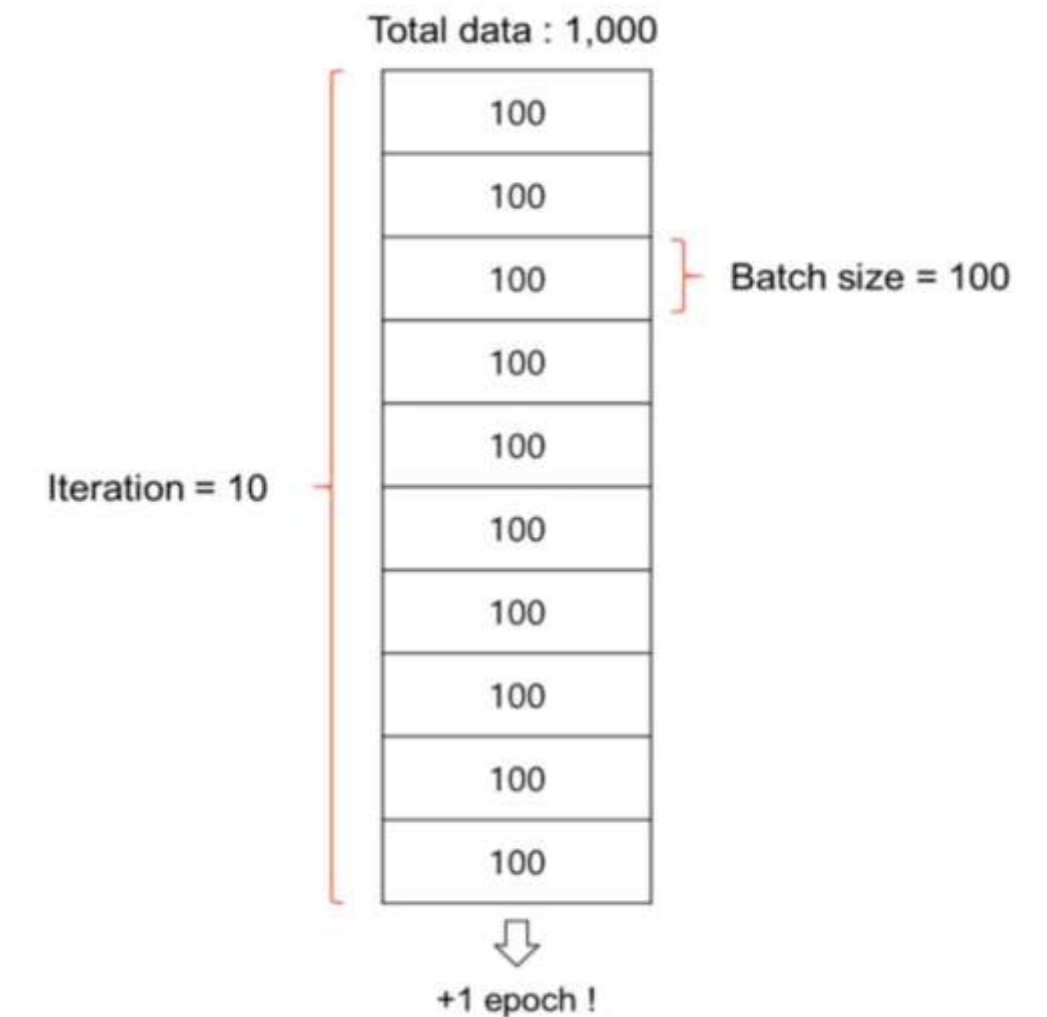
Learning Rate

- 1. Definition:** The learning rate (often denoted as η (*eta*)) is a scalar value that controls the magnitude of the updates applied to the model parameters (weights and biases) during each iteration of the optimization algorithm, such as gradient descent.
- 2. Impact on Training:**
 - 1. Convergence:** A high learning rate can lead to faster convergence because it allows the model to quickly traverse the parameter space. However, if the learning rate is too high, it may cause the optimization process to oscillate or diverge.
 - 2. Stability:** A low learning rate tends to result in a more stable training process, but it may require more iterations to converge to the optimal solution.



Batch Size and Epoch

1. **Definition:** The batch size represents the number of samples processed by the model in a single forward and backward pass during training. Instead of updating the model's parameters after every individual sample (which can be computationally expensive), training is typically performed on smaller subsets or batches of the training data.
2. **Impact on Training:**
 1. **Computational Efficiency:** Larger batch sizes can leverage parallelism and hardware optimizations to process multiple examples simultaneously, resulting in faster training times, especially on GPUs.
 2. **Generalization and Convergence:** Smaller batch sizes can lead to faster convergence and potentially better generalization since the model's parameters are updated more frequently and with more diverse examples. However, extremely small batch sizes may introduce instability and increase training time per epoch due to reduced computational efficiency.
 3. **Memory Constraints:** The choice of batch size is also influenced by memory constraints, as larger batch sizes require more memory to store activations and gradients during backpropagation.



Script Code Notebook Pembuatan Project

- **GitHub Repository Project TA**
<https://github.com/FauzanAriyatmoko/Sarung-Tangan-Pintar-Tunawicara>
- **Model Jaringan Saraf Tiruan (ANN)**
<https://colab.research.google.com/drive/1jnSlSHaYvjGA2fJxAw0yc3IL4370MPeQ?usp=sharing>
- **Eksplorasi Data Mendalam SIBI**
<https://colab.research.google.com/drive/1qYPiJHtNww7mb6p4HwNJgMugFhhX-BEQ?usp=sharing>
- **Perhitungan contoh Jaringan Saraf Tiruan Sederhana untuk Klasifikasi Data**
(1) <https://colab.research.google.com/drive/1tTZxnHsGgSjXe3gOncqVsfMBb1cYBF?usp=sharing>
(2) <https://colab.research.google.com/drive/1NuI28VXx-qcxJ2FYSFJ9yJe9KTPzFdRH?usp=sharing>
- **Percobaan perbandingan pada beberapa algoritma ML/DL lainnya**
<https://colab.research.google.com/drive/1jnRkjinJOawmAvWUGjhgX0G78RhTWwgVj?usp=sharing>

Arsitektur Model ANN

